

DE0-NANO

The DE0-Nano board is a compact and versatile FPGA development platform designed by Terasic Technologies

Key Features

FPGA Device: Cyclone IV EP4CE22F17C6N

- 22,320 Logic Elements (LEs)
- 594 Kbits embedded memory
- 66 embedded multipliers

Memory:

- 32MB SDRAM
- 2KB EEPROM (I²C)

User Interfaces:

- 8 Green User LEDs
- 2 Push Buttons
- 4 DIP Switches

Connectivity:

- USB-Blaster II (on-board JTAG and configuration interface)
- Mini-USB for power and programming
- 2x 40-pin GPIO headers (expandable to external modules)

Sensors:

- Accelerometer (3-axis, Analog Devices ADXL345)

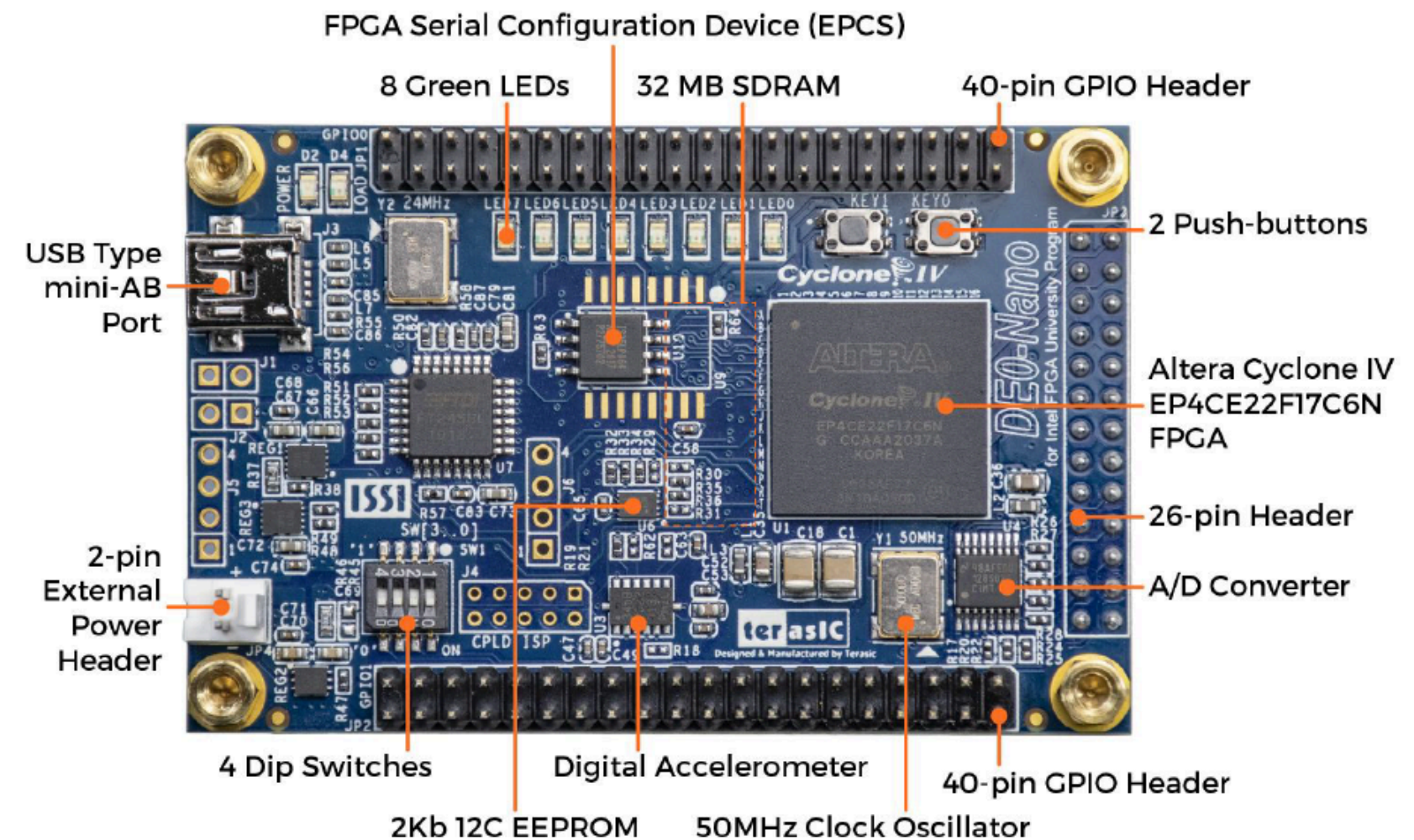


Figure 2-1 The DE0-Nano Board PCB and component diagram (top view)

Development Tools:

- Quartus Prime (for synthesis and programming)
- ModelSim (for simulation)

IN-BUILT CLOCK AND ADC



Figure 3-14 Block diagram of the clock distribution

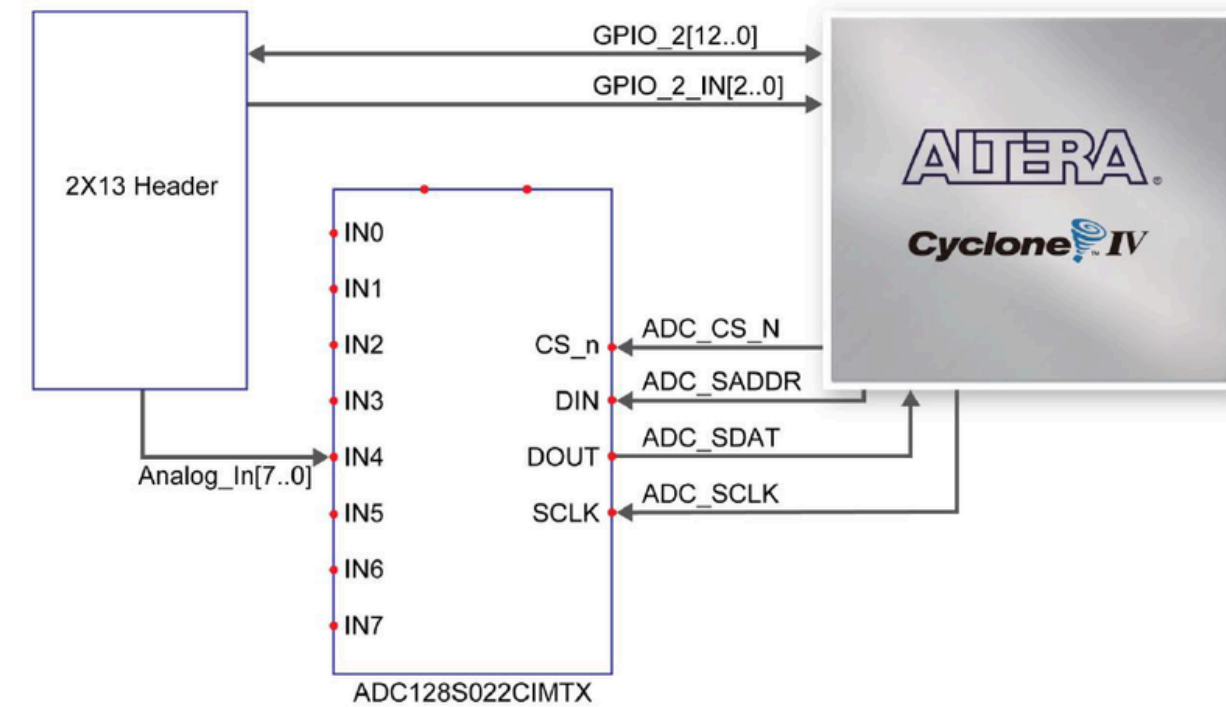


Figure 3-11 Wiring for 2x13 header and A/D converter

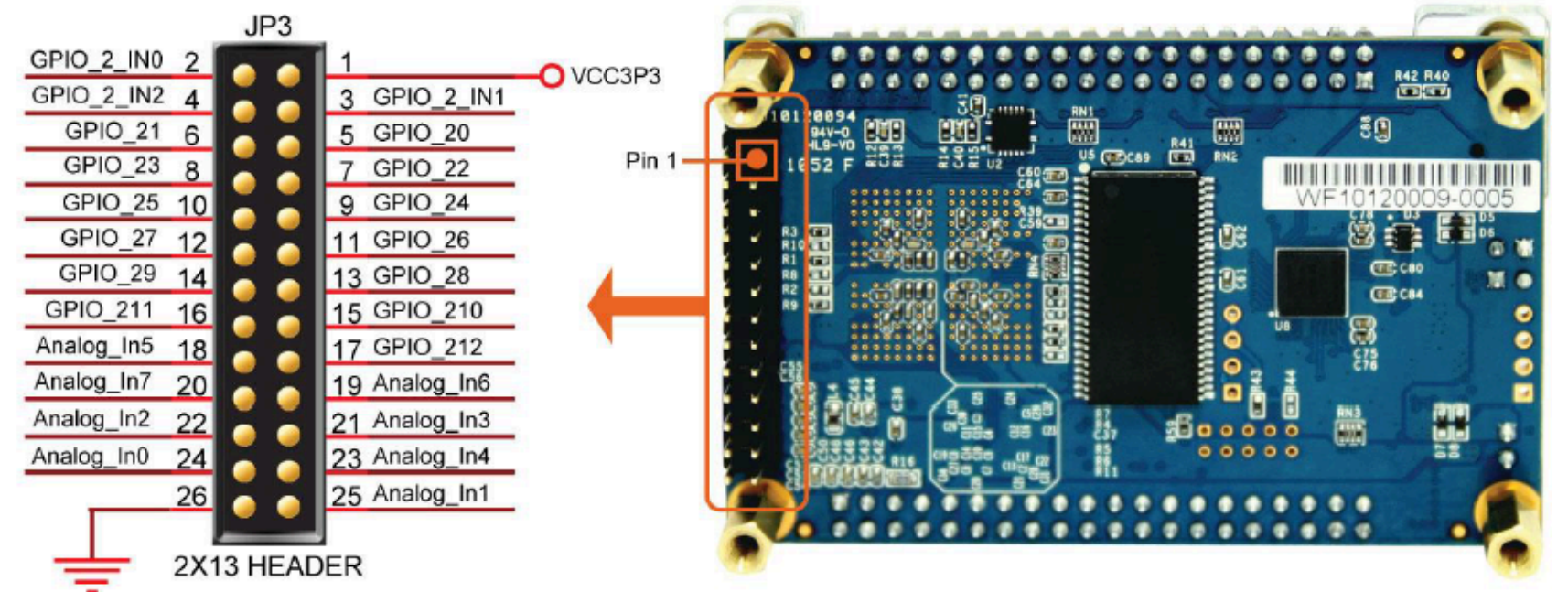


Figure 3-10 Pin distribution of the 2x13 Header

SIMULATION SETUP

These module is implemented on the fgpa board

Select_cell.v
module



Brief overview of setup

Select_cell: This Module is used for controlling the peripherals for selecting a cell in the NVM crossbar as provided by the input row address and column adders.

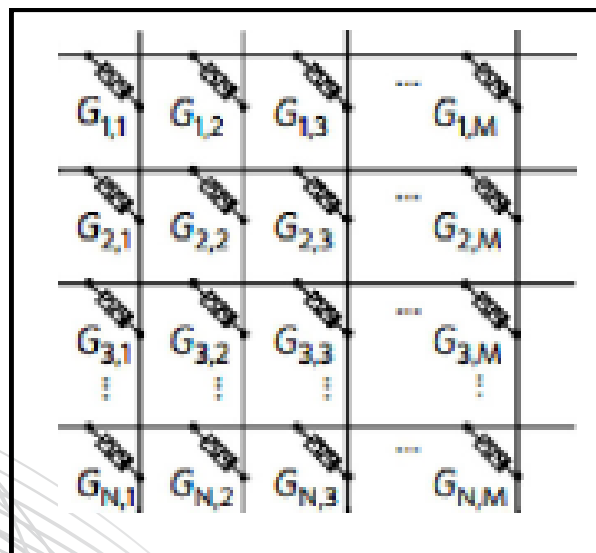
Peripherals: This includes an analog switch, decoder, analog multiplexer, and ADC.

A digital replica has been created via a test bench for simulation purposes, namely, Mock-analog switch, Mock-analog mux and decoder.

The Fpga controle the periferals to read and write the nvm crossbar

peripherals

NVM Crossbar



Periferal directly interface with NVM crossbar

SIMULATION SETUP

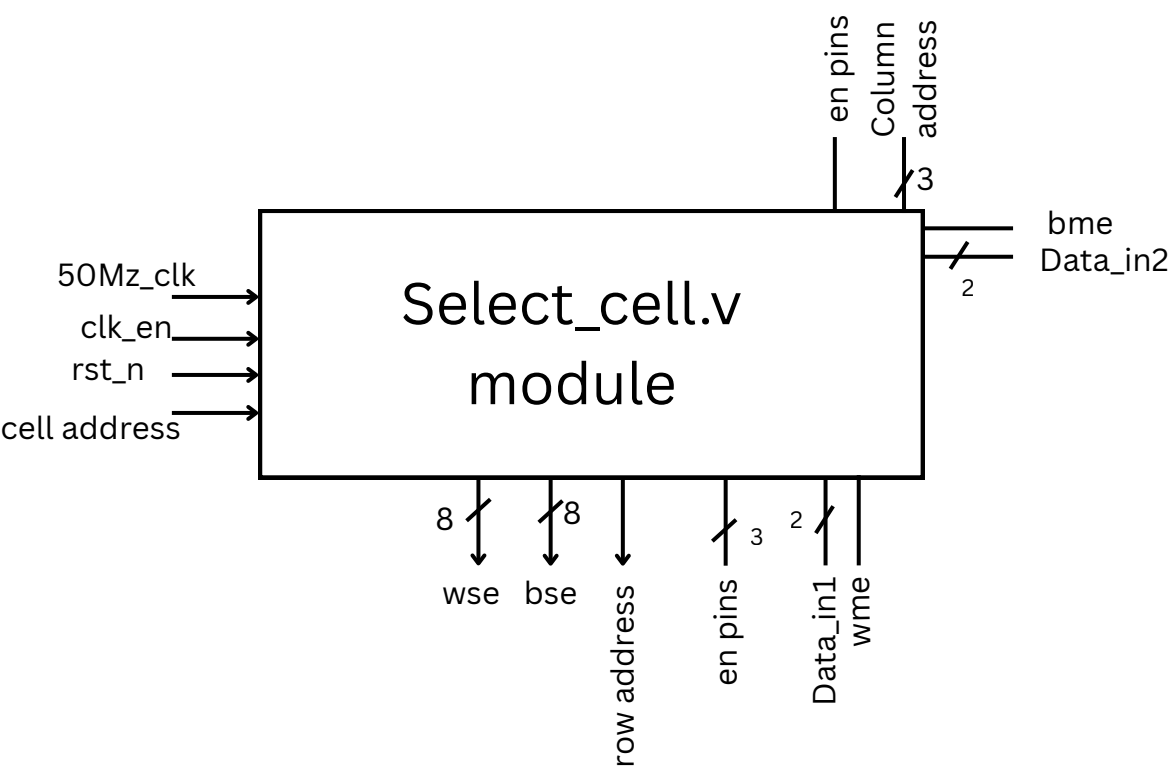
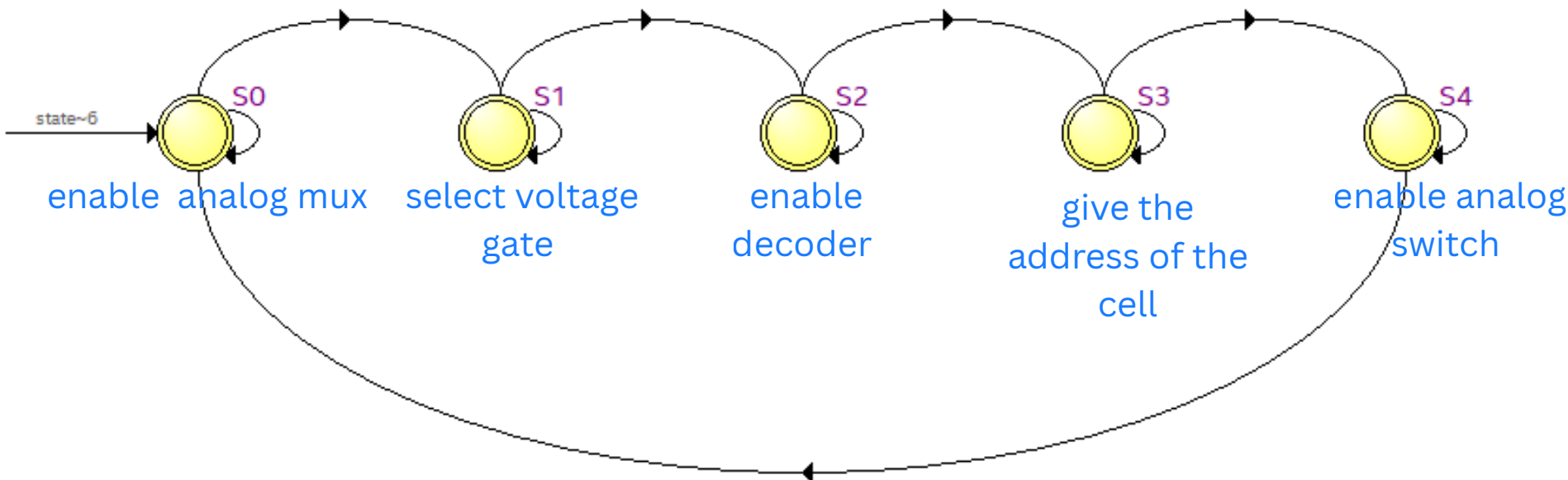


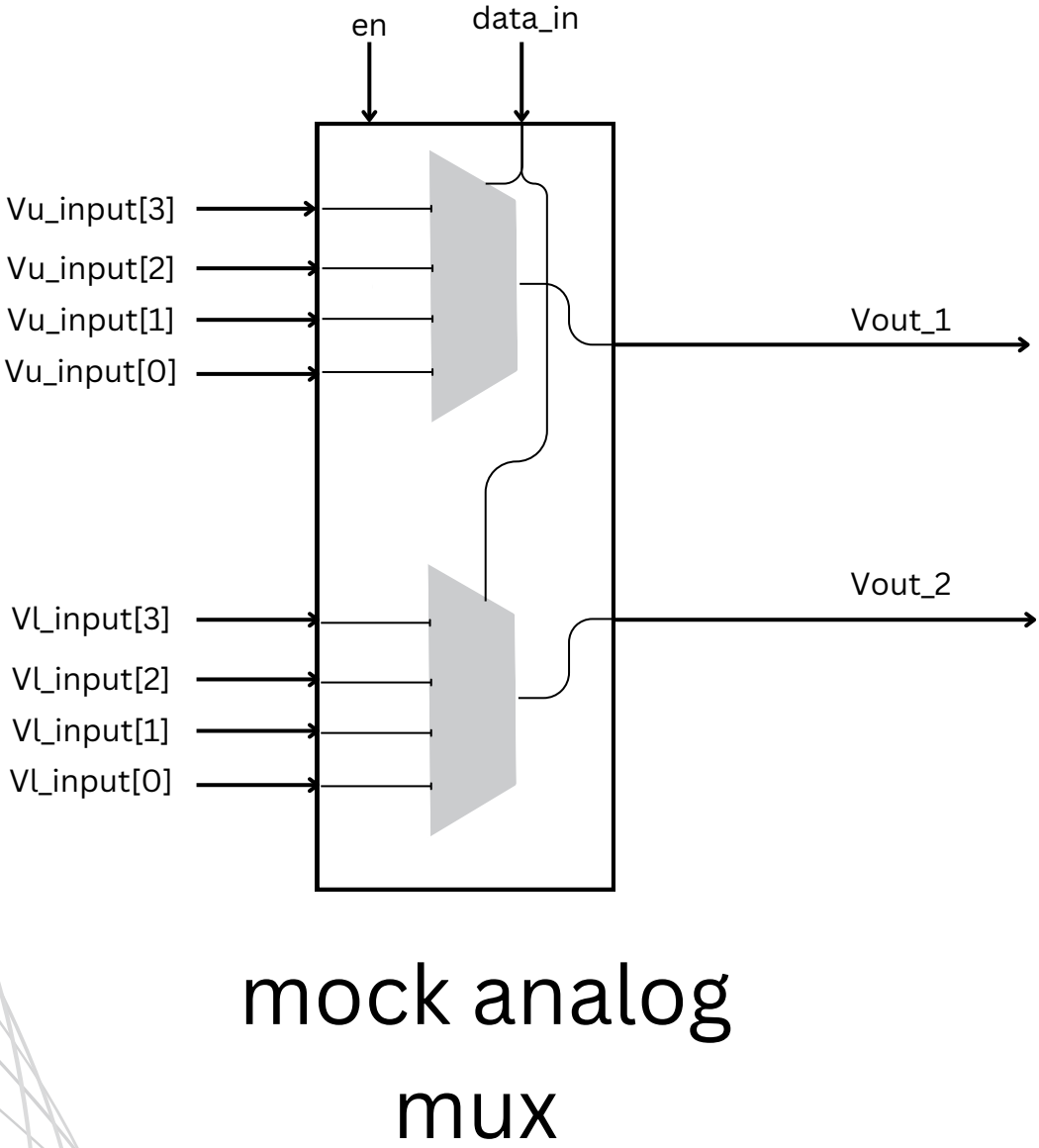
Fig: Select cell module

	Source State	Destination State	Condition
1	S0	S1	(clk_en)
2	S0	S0	(!clk_en)
3	S1	S2	(clk_en)
4	S1	S1	(!clk_en)
5	S2	S2	(!clk_en)
6	S2	S3	(clk_en)
7	S3	S3	(!clk_en)
8	S3	S4	(clk_en)
9	S4	S4	(!clk_en)
10	S4	S0	(clk_en)

Signal	Type	Description
wse	output [7:0]	Enable signals for wordline-side analog switches (mock analog switches driven by decoder)
bse	output [7:0]	Enable signals for bitline-side analog switches (mock analog switches driven by decoder)
row_add	output [2:0]	Wordline row address – used by row decoder to activate the desired wordline
column_add	output [2:0]	Bitline column address – used by column decoder to activate the desired bitline
we1_n, we2_n	output	Write enable pins for wordline drivers (active low)
be1_n, be2_n	output	Write enable pins for bitline drivers (active low)
wl_rdw	output	Direction control for wordline driver (e.g., 0 = read, 1 = write)
bl_rdw	output	Direction control for bitline driver (e.g., 0 = read, 1 = write)
datain_1	output [1:0]	Select lines for channel selection of analog MUX for wordline side
datain_2	output [1:0]	Select lines for channel selection of analog MUX for bitline side
wme	output	Enable for wordline MUX (analog MUX control)
bme	output	Enable for bitline MUX (analog MUX control)
clk	input	50 MHz system clock driving all sequential logic (i.e., FSM timing and counter)
clk_en	input	Clock enable signal — when high, allows FSM state transitions; when low, FSM pauses
rst_n	input	Active-low asynchronous reset — resets the FSM and output control logic
W	input [2:0]	Wordline (row) address – part of the cell address to be selected
B	input [2:0]	Bitline (column) address – part of the cell address to be selected
Data_input	input [1:0]	Data to be sent to the analog MUXes (selects channel at input side)
counts	input [2:0]	Number of clock cycles to wait during ADC/conversion phase (in S4)



SIMULATION SETUP



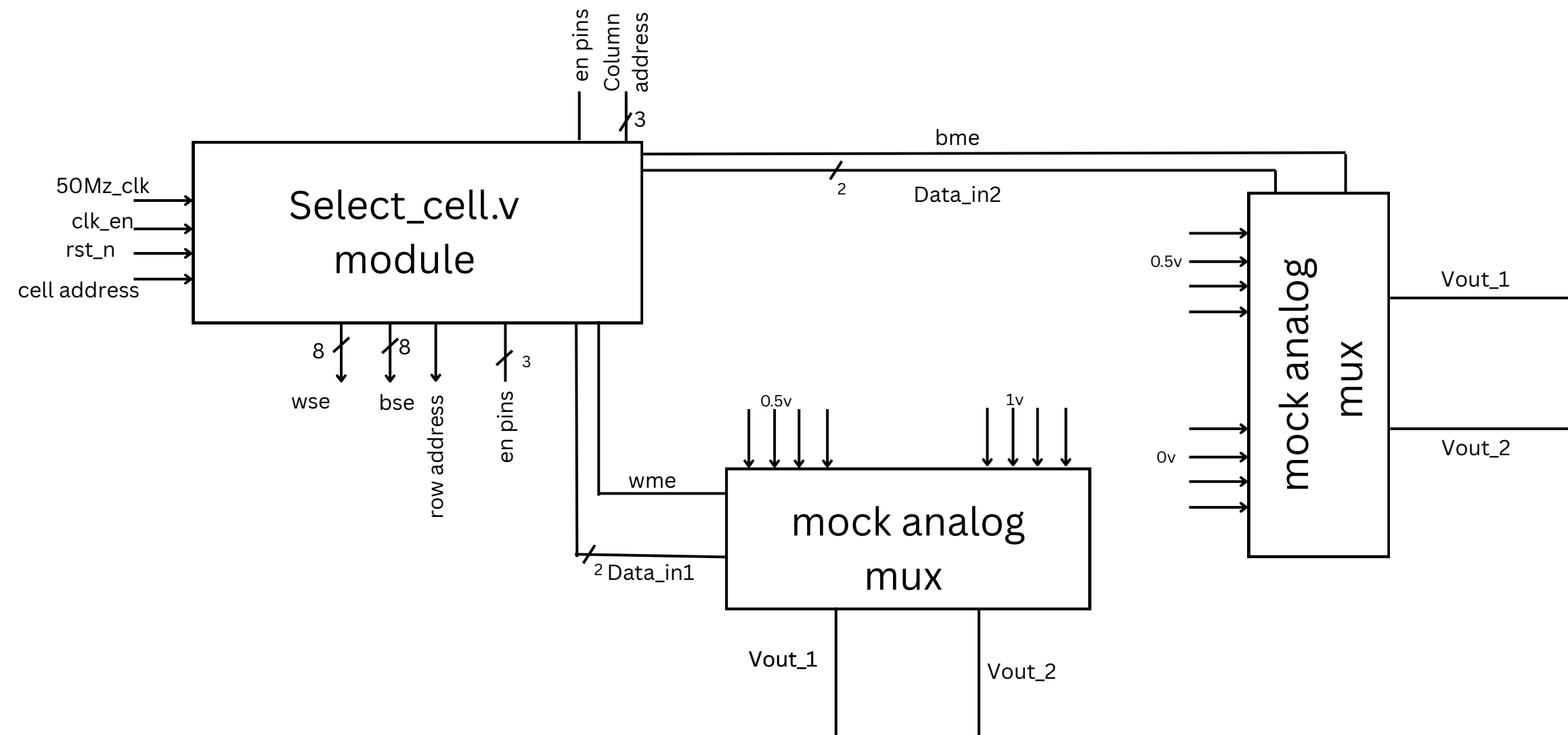
Mock analog mux: In Verilog, you cannot model real, continuous analog voltage behavior. So, in RTL, we create a mock version that digitally mimics the control behavior of an analog MUX

Pin Description:
Vu_input: Upper input voltage
VL_input: lower input voltage
vout_1: output of the upper mux
vout_2: output of the lower mux
data_in: It is select line.
en: to enable the ic.

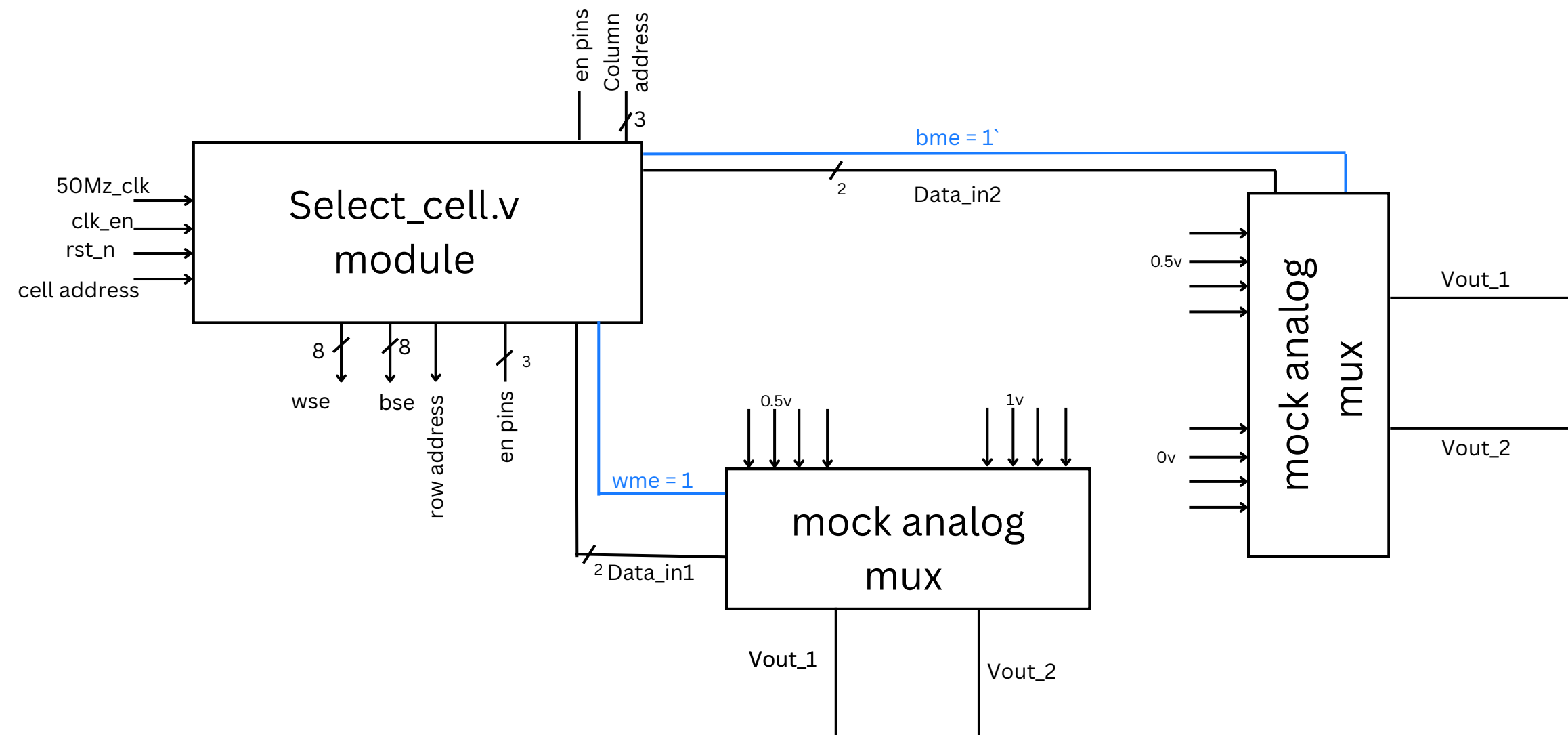
Truth Table

en	data_in	Vout_1	Vout_2
1	XX	High z	High z
0	00	Vu_input[3]	Vu_input[3]
0	01	Vu_input[2]	Vu_input[2]
0	10	Vu_input[1]	Vu_input[1]
0	11	Vu_input[0]	Vu_input[0]

SIMULATION SETUP

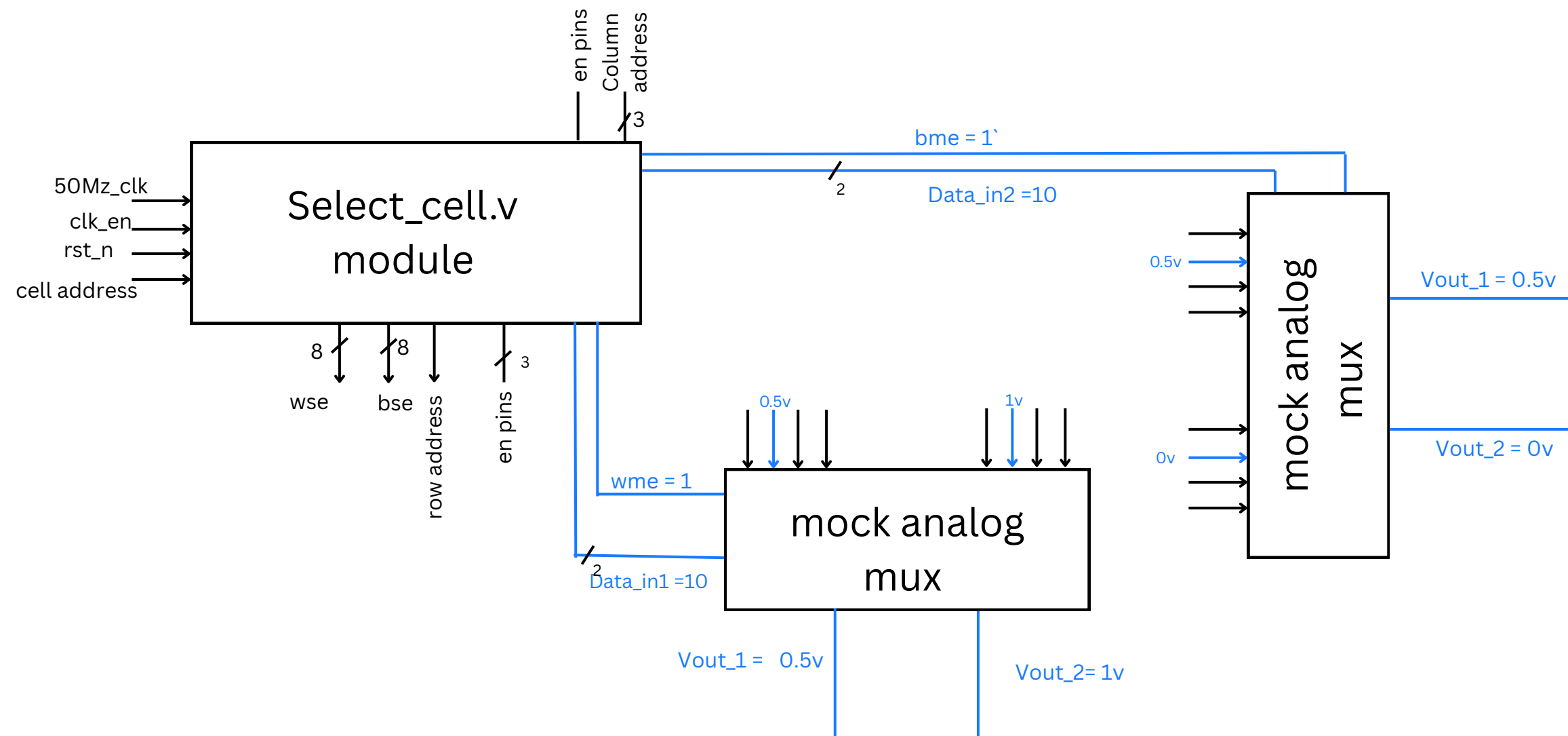


SIMULATION SETUP



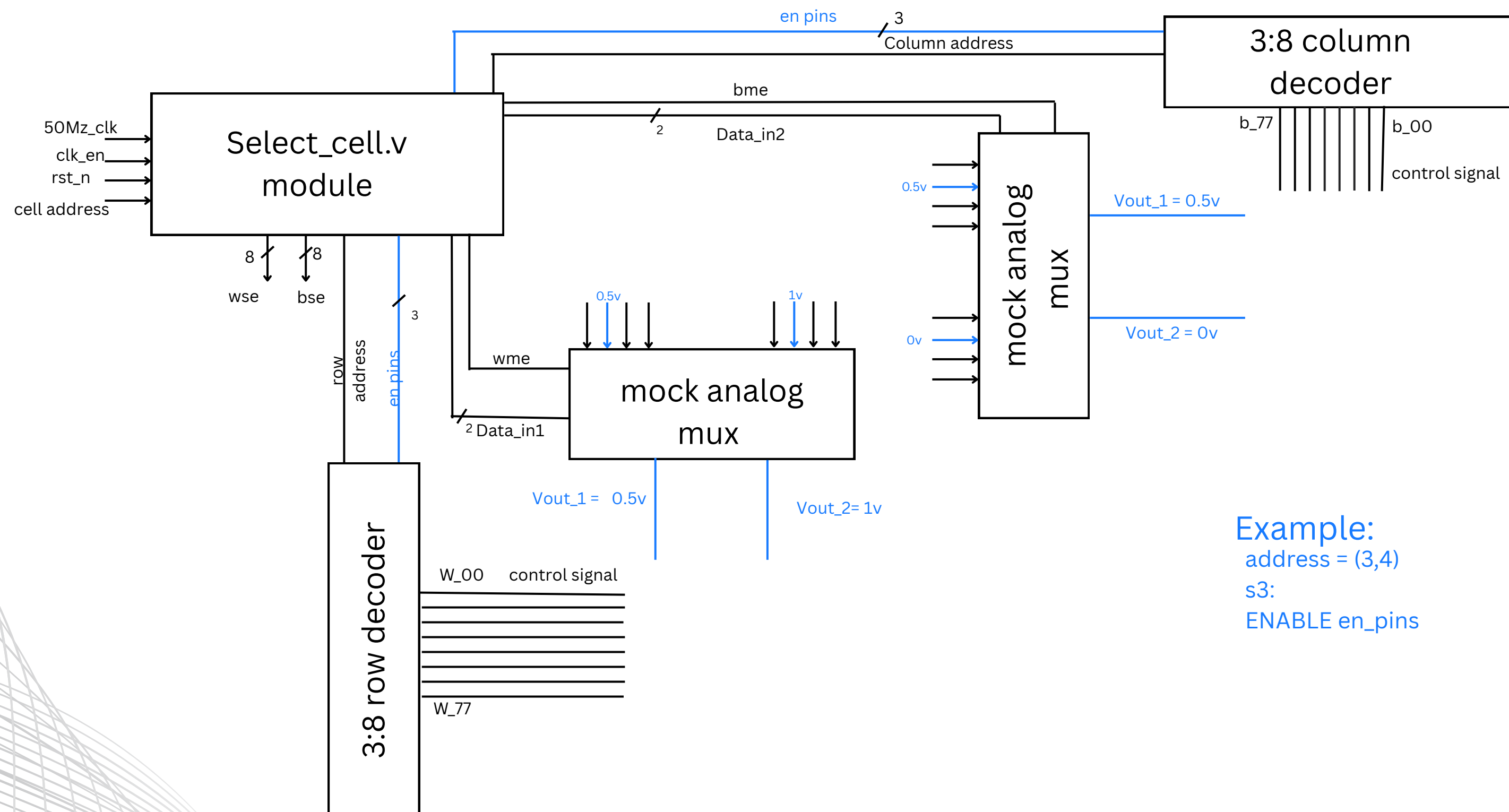
Example:
address = (3,4)
s0:
wme = 1
bme = 1

SIMULATION SETUP

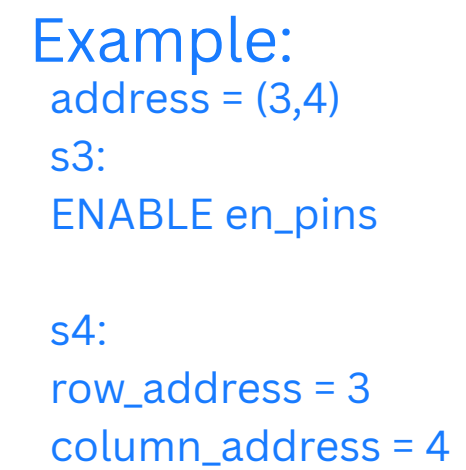


Example:
address = (3,4)
s0:
wme = 1
bme = 1
s2:
data_in1,2 = 10

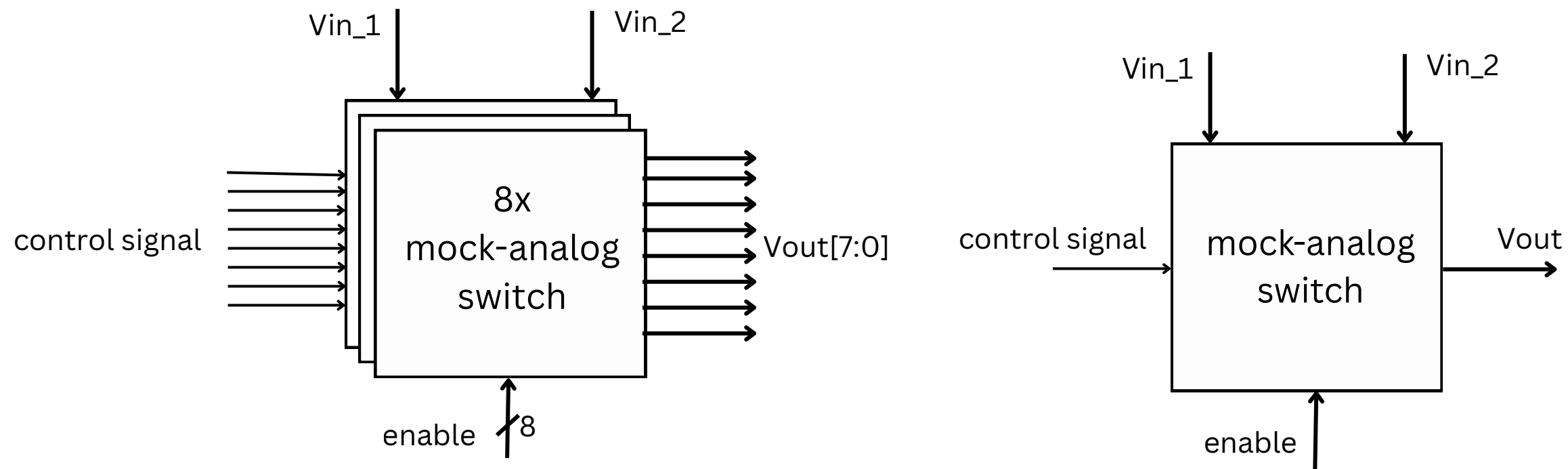
SIMULATION SETUP



Example:
address = (3,4)
s3:
ENABLE en_pins



SIMULATION SETUP



Mock analog switch: In Verilog, you cannot model real, continuous analog voltage behavior. So, in RTL, we create a mock version that digitally mimics the control behavior of an analog switch

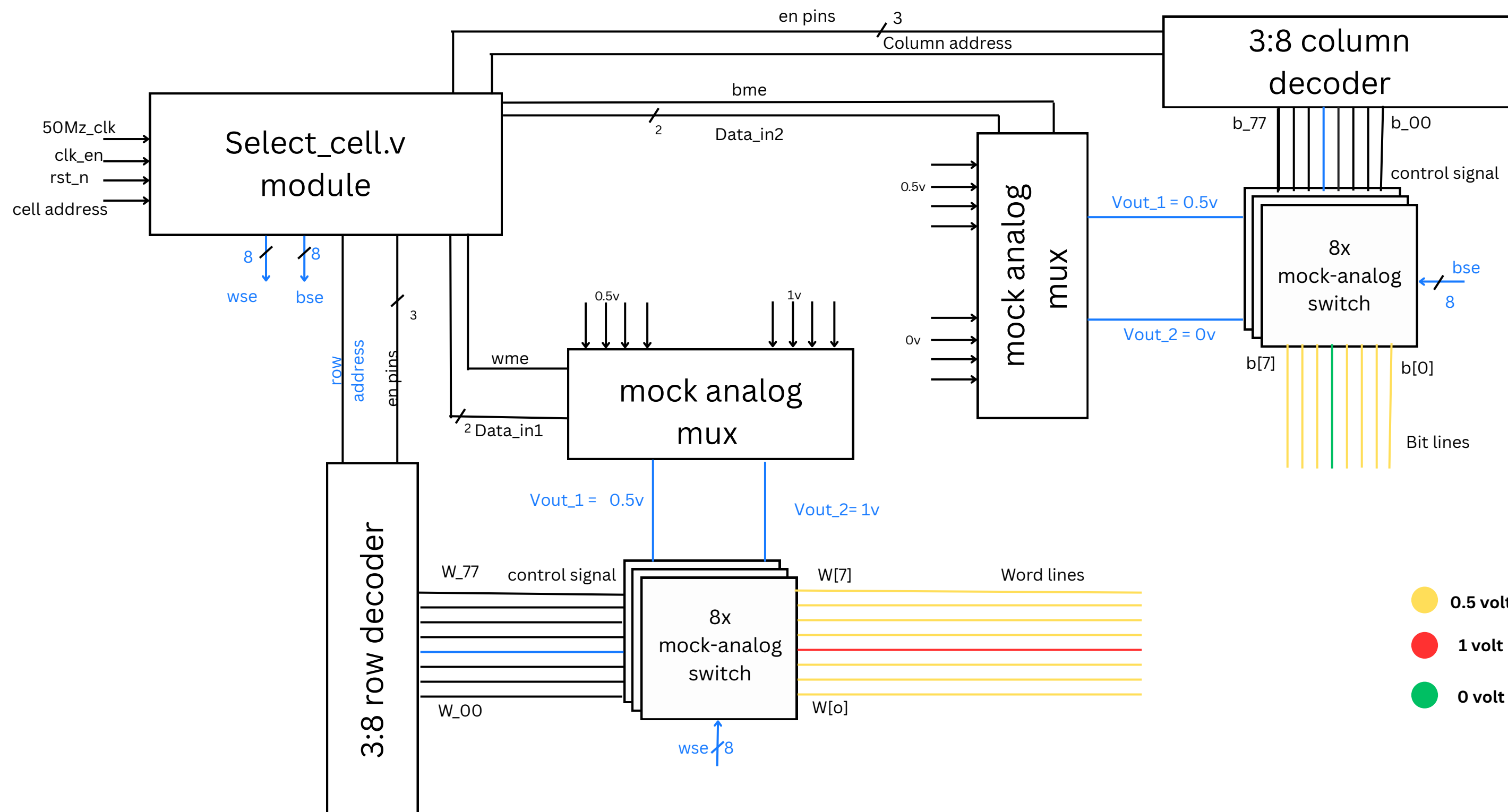
Pin description

Pin Name	Direction	Description
Vin_1	Input	First analog input signal to the switch
Vin_2	Input	Second analog input signal to the switch
control_signal	Input	Select line: chooses which Vin gets connected to Vout
enable	Input	Master enable: allows switching to happen when HIGH
Vout	Output	Output signal — follows selected input if enabled

Truth table

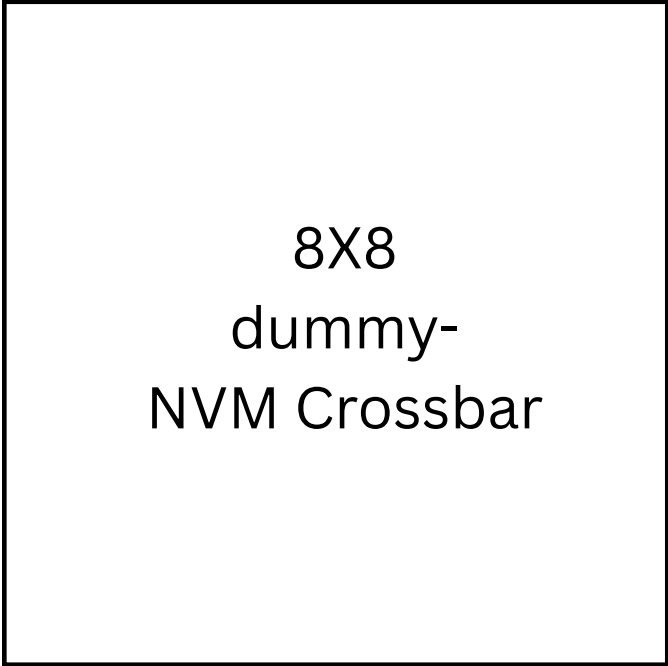
enable	Control Signal	Vout	Behavior
0	X	High-Z / 0	Switch is OFF
1	0	Vin_1	Connect Vin_1 to Vout
1	1	Vin_2	Connect Vin_2 to Vout

SIMULATION SETUP



Example:
address = (3,4)
s4:
wse = 8'b1
bse = 8'b1

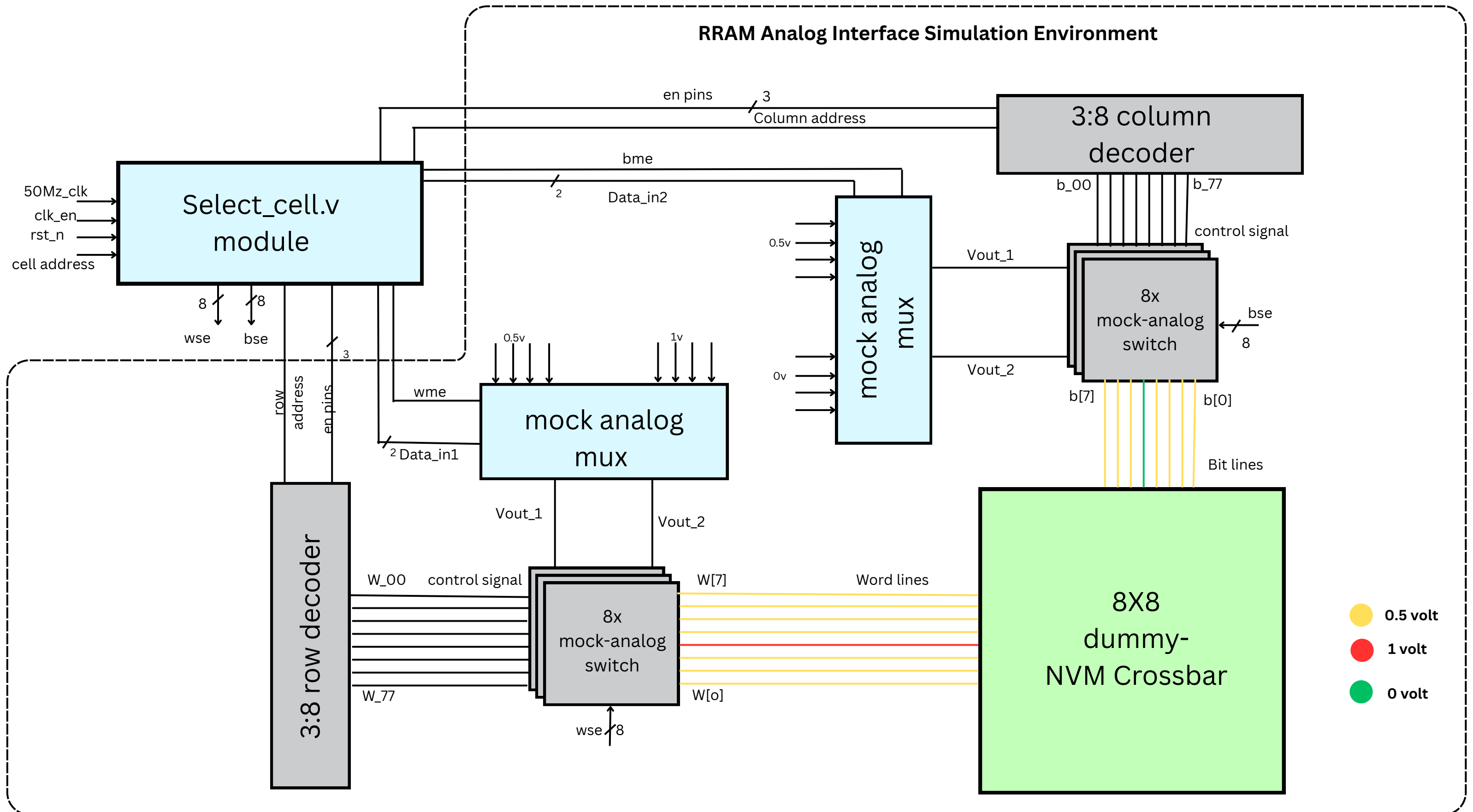
SIMULATION SETUP



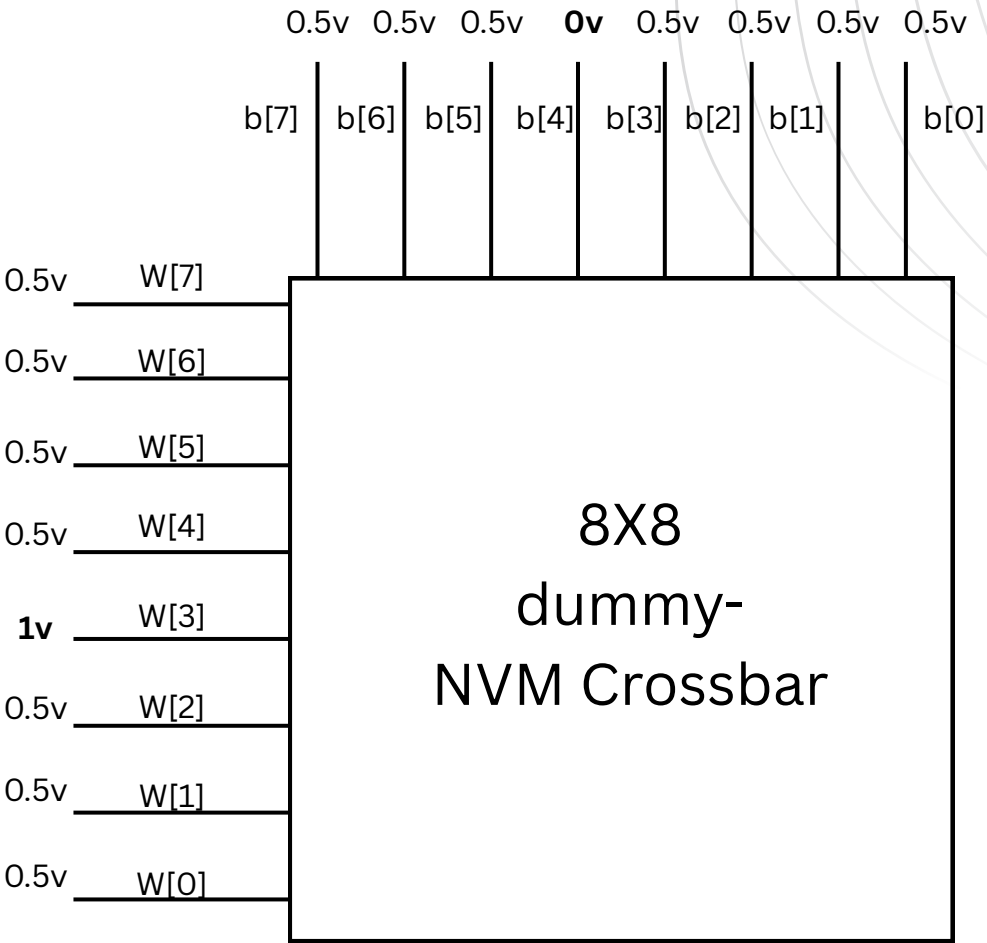
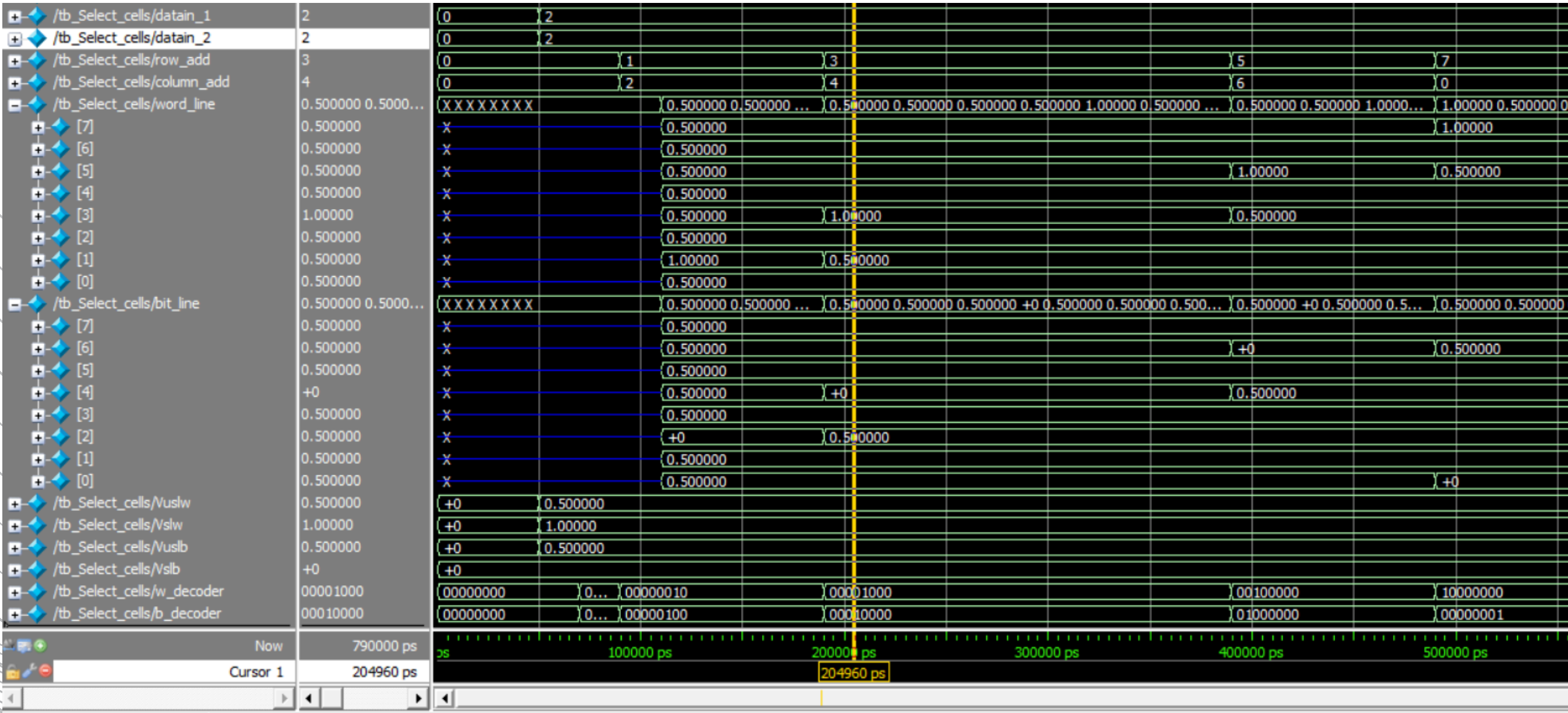
8X8
dummy-
NVM Crossbar

This is an empty module with just input and output pins, no logic just for simulation pourpouse .

SIMULATION SETUP

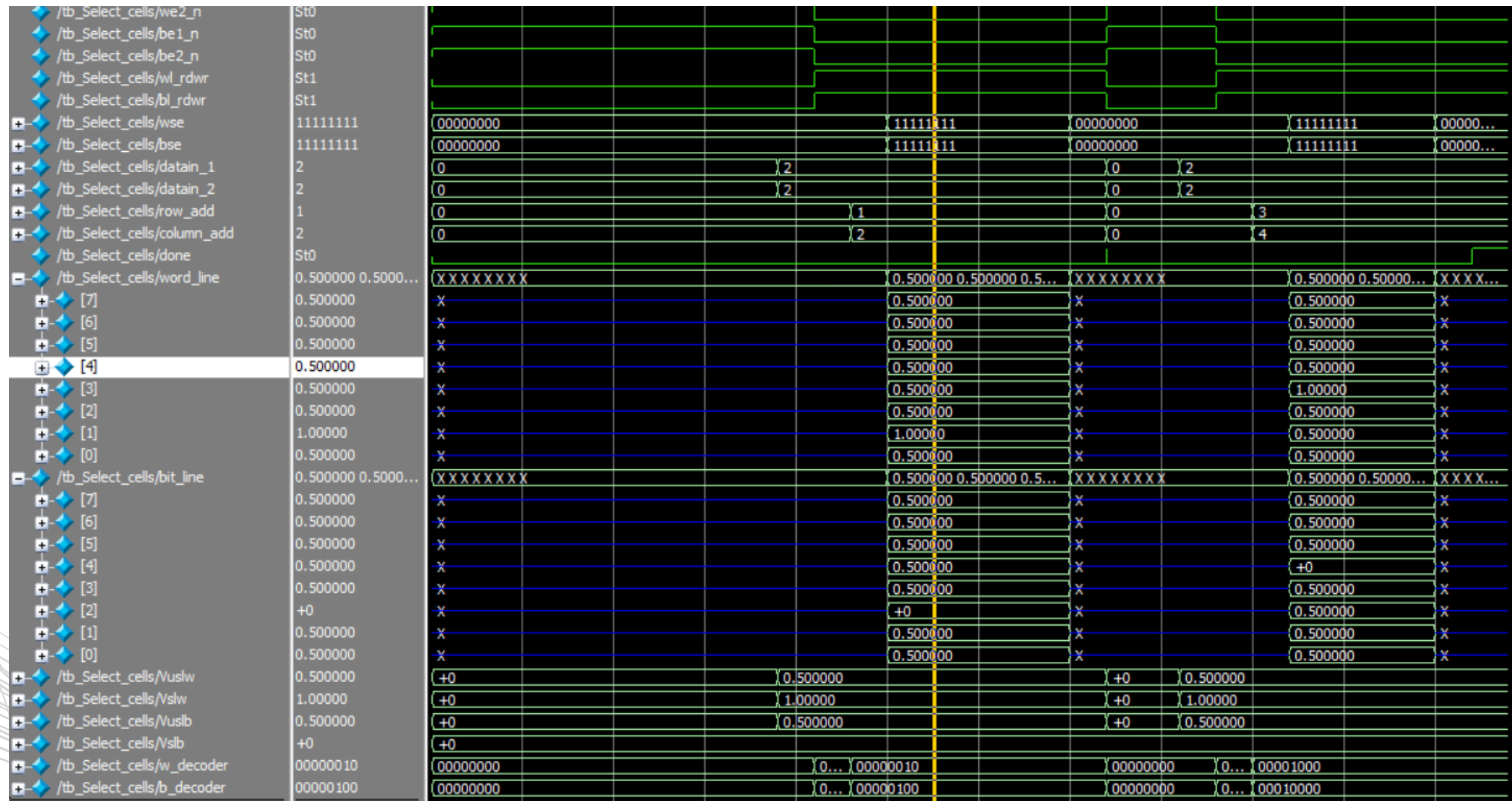


RESULTS

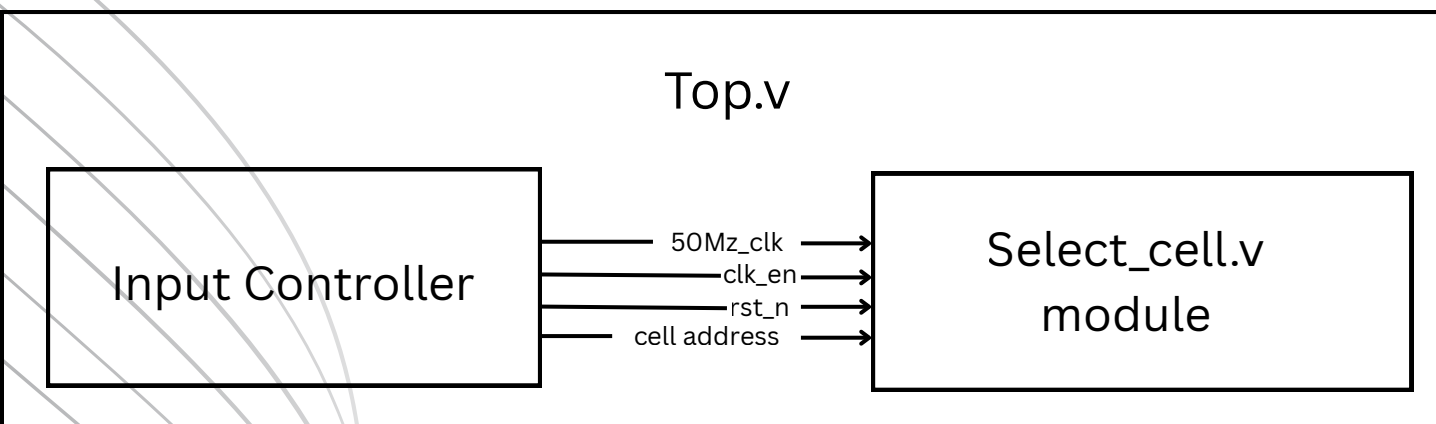


NEW ADJUSTMENTS

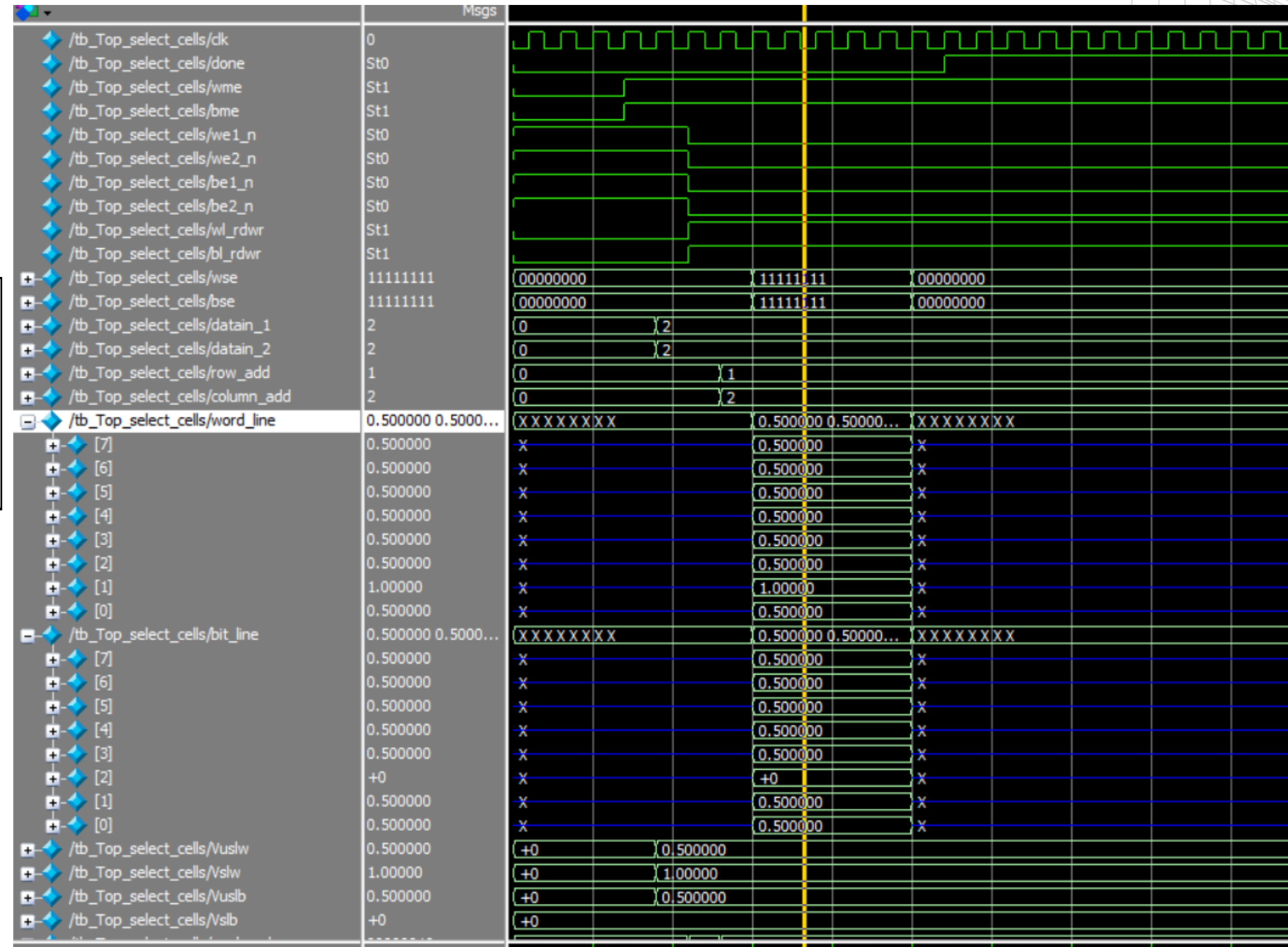
- **State 5:** Added a new state S5 in the FSM of select_cel, an idle state where it produces a signal “done” and remains in the state until the “Restart” signal is made high.
- **Done:** an output signal from select_cell, which implies that the operation, i.e selecting the cell, is done, which can be later used and SOC (start of conversion of ADC)
- **Restart:** To return back to the S0 state for a new selecting operation.
- **Count :** It's also an input to the Select_cell. It determines the pulse width of the select pulse, effectively turning the selection operation into a pulsing operation.



EXPREMENT SETTINGS



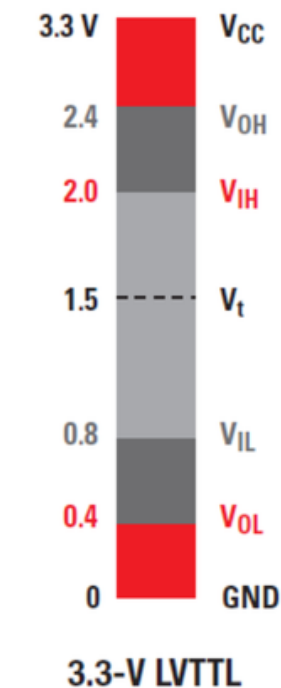
- To provide input to select_cell, another module, Top.v is created. It supplies inputs to select_cell.v automatically, reducing the need for manual intervention and thereby minimizing human error.



SYNTHESIS DETAILS

Pin assignments over view

- Pin assignments were done before synthesis.
- The done signal was connected to LED[0] on the DE0-Nano board.
- The clk signal was assigned to PIN_R8, which is the internal 50 MHz clock oscillator.
- I/O standard was set to 3.3V LVTTL.
- The current strength was set to 4mA except done (8ma).
- Slew rate was configured as 2 (slow).



Project Overview

Parameter	Value
Project Name	exp_testing
Top-Level Entity	Top
Target Board	DE0-Nano
FPGA Device	Cyclone IV E — EP4CE22F17C6
Clock Frequency	50 MHz (20 ns period)
Clock Source	Internal oscillator (CLOCK_50, pin R8)
Quartus Version	20.1.1 Lite Edition

Resource Utilization

Total logic elements	26 / 22,320 (< 1 %)
Total registers	49
Total pins	36 / 154 (23 %)
Total virtual pins	0
Total memory bits	0 / 608,256 (0 %)
Embedded Multiplier 9-bit elements	0 / 132 (0 %)
Total PLLs	0 / 4 (0 %)

TIMING ANALYSIS REPORT

worst-case timing slacks

Metric	Value
Setup Slack	8.183 ns
Hold Slack	0.187 ns
Recovery Slack	14.084 ns
Removal Slack	0.577 ns
Minimum Pulse Width	9.274 ns

Slack	From Node	To Node
8.183	Select_cell:uut[column_add[1]	column_add[1]

Column_add[1] is the critical path with the worst case slack.

Slack = Required time – Actual arrival time
Therefore Slack should be positive. As in the design all worst case slack are positive so the design have met its timing requirments.

Fmax Summary

Model	Fmax
slow 1200mv 85c	84.62 MHZ
slow 1200mv 0c	90.47 MHZ

Target Frequency = 50 MHz

PRECAUTIONS WHILE ON HARDWARE

- Human error while coding can cause wrong voltage
- Solved by making the program hardcoded and tested in ModelSim.
- Logic level and driving current of FPGA should be set properly before programming
- Connect the hardware pins properly as per the pin assignments
- Verify pin connections multiple times
- Applied voltage should be set properly before applying to the hardware
- It should not be more than 3.3V.
- For testing, we will use not more than 1V so it's safe
- Carefully connect the VCC and GND to the ICs

NEXT

- Hardware Setup and Verification | Tentative time requirement: 1 to 2 days

1. Verify the hardware to ensure each component is functioning properly.
2. Implement the code and verify its functionality by checking the wordline and bitline voltages using an oscilloscope or multimeter.

- Modify for Partial Bias Scheme (Read/Write) | Tentative time requirement: 3 days

Modify the code for reading and writing each cell at a time in nvm crossbar.

- Implement weight mapping and MAC operation | Tentative time requirement: 7 days

1. Modify the code to map weight in the NVM crossbar.
2. Implement Multiply multiply-accumulate operation in NvM crossbar using random weight.

READ/WRITE PULSE

We demonstrate that the model can reproduce experimentally observed Set and Reset transient across a range of timescale (100 ns – 1 s)

V. Saraswat et al.

Why do we need a short voltage pulse?

- Ensures precise switching without overdriving the cell.
- Reduces Joule heating and prevents thermal damage.
- Limits the sneak path current duration in crossbar arrays.
- Improves endurance by minimizing stress per cycle.
- Enables faster write and erase operations.
- Prevents unintentional switching in nearby cells.
- Lowers overall energy consumption during switching.

Disasvantage!!!

- Due to delays of various ICs, how can we drive a precise 100 ns pulse at high frequency, like 50 MHz?
- How can the built-in ADC in the DEO-Nano sample the 100 ns read pulse?

IN the expreiment the reada/write pulse was simulated for only 100 ns .

FREQUENCY CALCULATION

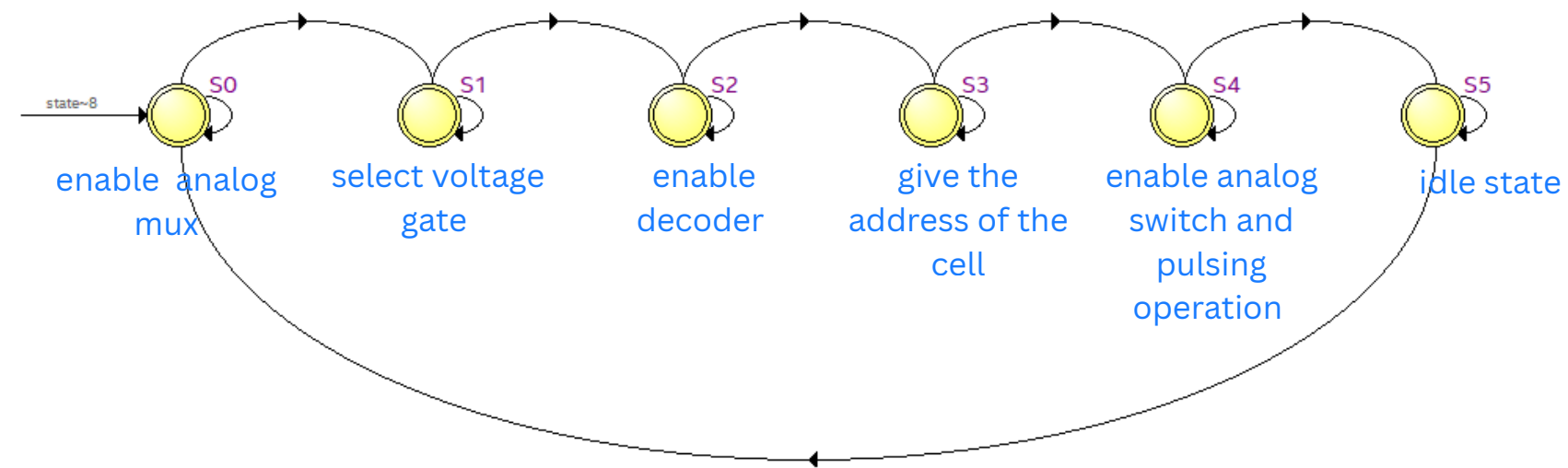
Parameters

- **t_{on}(mux_en)**: Time required for the analog multiplexer to become enabled.
- **t_{on}(sw_en)**: Time required for the analog switch to become enabled.
- **t_{trans1}**: Transition time of the analog multiplexer.
- **t_{trans2}**: Transition time of the analog switch.
- **t_{off}(mux_en)**: Time required for the analog multiplexer to disable after deasserting the enable signal.
- **t_{off}(sw_en)**: Time required for the analog switch to disable after deasserting the enable signal.
- **t_p**: propagation delay of decoder
- **pw** : pulse width of the read/write pulse

Assumption

t_p is neglected as it is very less compared to the other time parameters

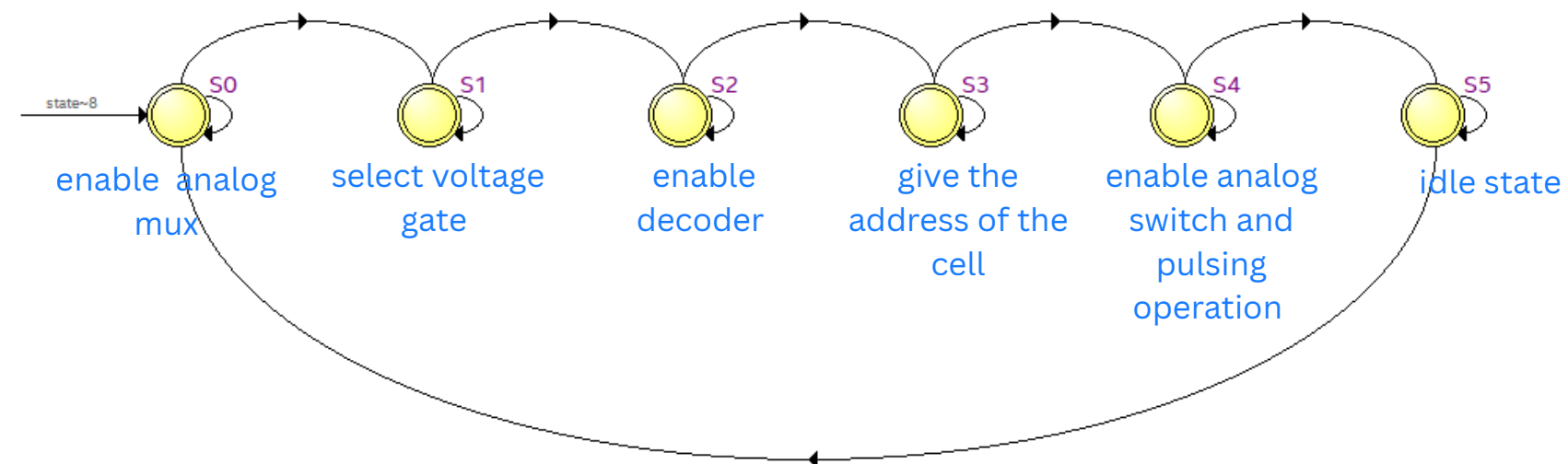
FREQUENCY CALCULATION



Condition 1

- Output of the mux should be settled before the pulsing operation

FREQUENCY CALCULATION



Before these

```
S4: begin
    // Enable
    wse <= 8'b11111111;
    bse <= 8'b11111111;

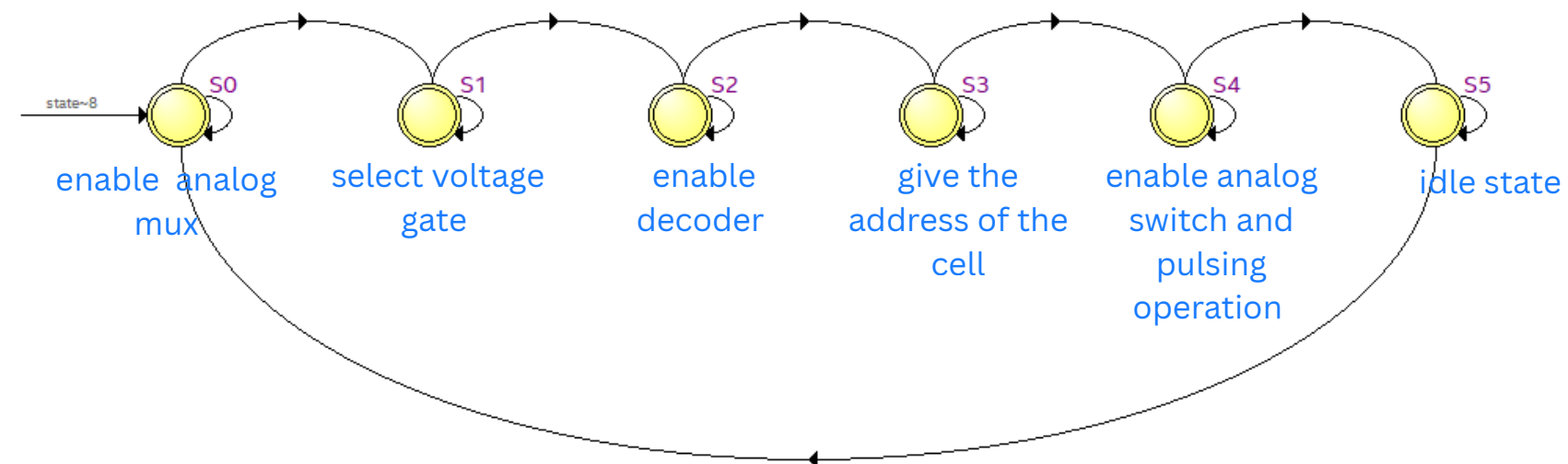
    if (count_reg < counts) begin
        count_reg <= count_reg + 3'd1;
        state <= S4; // stay in S4
    end else begin

        wse <= 8'b0;
        bse <= 8'b0;

        count_reg <= 0;
        state <= S5; // proceed to S5 after wait
    end
end
```

STATE 4

FREQUENCY CALCULATION



Before these

```
S4: begin
    // Enable
    wse <= 8'b11111111;
    bse <= 8'b11111111;

    if (count_reg < counts) begin
        count_reg <= count_reg + 3'd1;
        state <= S4; // stay in S4
    end else begin

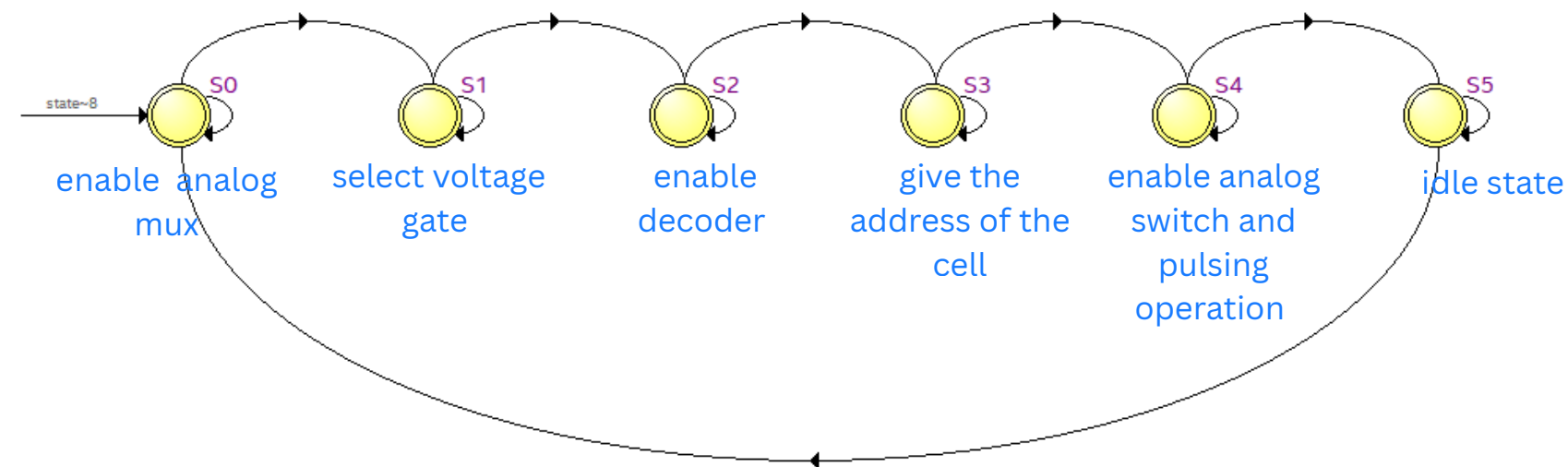
        wse <= 8'b0;
        bse <= 8'b0;

        count_reg <= 0;
        state <= S5; // proceed to S5 after wait
    end
end
```

$$t_{\text{on}}(\text{mux_en}) + t_{\text{trans1}} \leq (S1+S2+S3) + t_{\text{on}}(\text{sw_en})$$

STATE 4

FREQUENCY CALCULATION



Condition 2

- The pulsing should be completed before the analog switches are off again

Before these →

```
S4: begin
    // Enable
    wse <= 8'b11111111;
    bse <= 8'b11111111;

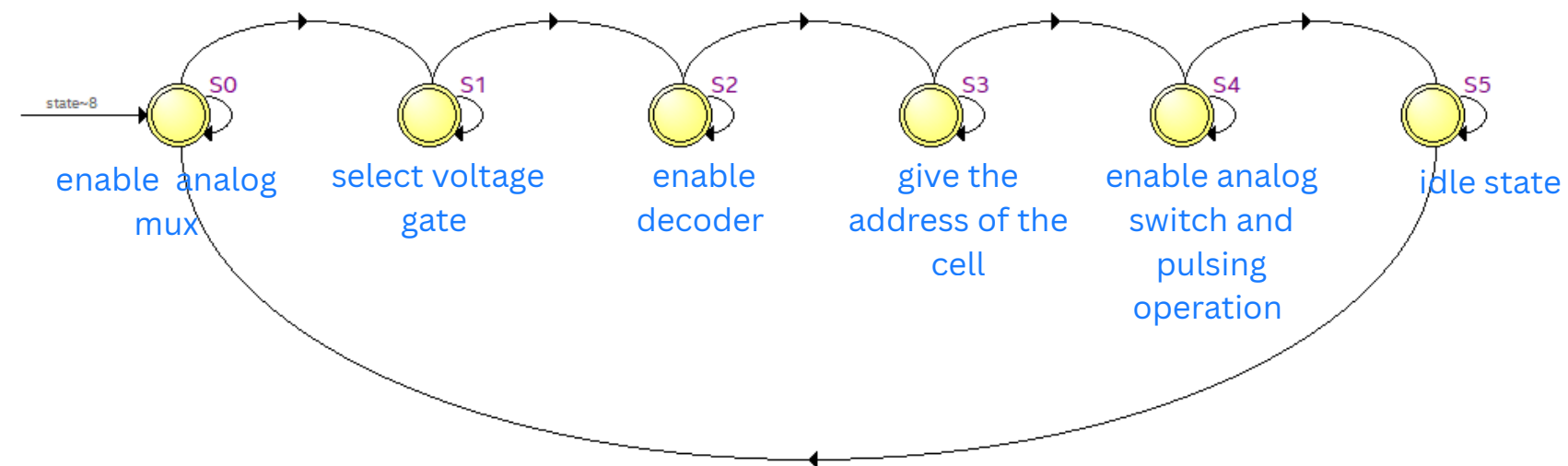
    if (count_reg < counts) begin
        count_reg <= count_reg + 3'd1;
        state <= S4; // stay in S4
    end else begin

        wse <= 8'b0;
        bse <= 8'b0;

        count_reg <= 0;
        state <= S5; // proceed to S5 after wait
    end
end
```

STATE 4

FREQUENCY CALCULATION



$$t_{trans2} \leq \text{Delay} - t_{on}(sw_en) + pw + t_{off}(sw_en)$$

Before these →

```
S4: begin
    // Enable
    wse <= 8'b11111111;
    bse <= 8'b11111111;

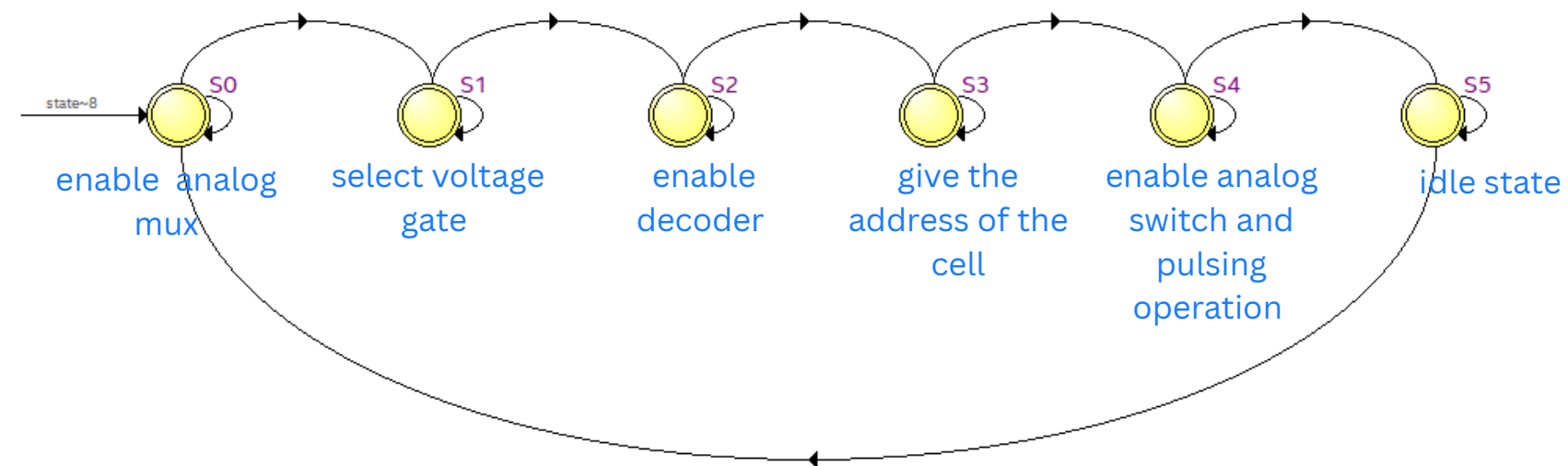
    if (count_reg < counts) begin
        count_reg <= count_reg + 3'd1;
        state <= S4; // stay in S4
    end else begin

        wse <= 8'b0;
        bse <= 8'b0;

        count_reg <= 0;
        state <= S5; // proceed to S5 after wait
    end
end
```

STATE 4

FREQUENCY CALCULATION



$$t_{\text{on}}(\text{mux_en}) + t_{\text{trans1}} \leq (S1+S2+S3) + t_{\text{on}}(\text{sw_en}) \dots\dots\dots(1)$$

$$t_{\text{trans2}} \leq \text{Delay} - t_{\text{on}}(\text{sw_en}) + \text{pw} + t_{\text{off}}(\text{sw_en}) \dots\dots\dots(2)$$

```
S4: begin
    // Enable
    wse <= 8'b11111111;
    bse <= 8'b11111111;

    if (count_reg < counts) begin
        count_reg <= count_reg + 3'd1;
        state <= S4; // stay in S4
    end else begin

        wse <= 8'b0;
        bse <= 8'b0;

        count_reg <= 0;
        state <= S5; // proceed to S5 after wait
    end
end
```

STATE 4

FREQUENCY CALCULATION

$$t_{on}(mux_en) + t_{trans1} \leq (S1+S2+S3) + t_{on}(sw_en) \dots\dots\dots(1)$$

$$t_{trans2} \leq Delay - t_{on}(sw_en) + pw + t_{off}(sw_en) \dots\dots\dots(2)$$

by solving these for Delay = n x t_clk we get,

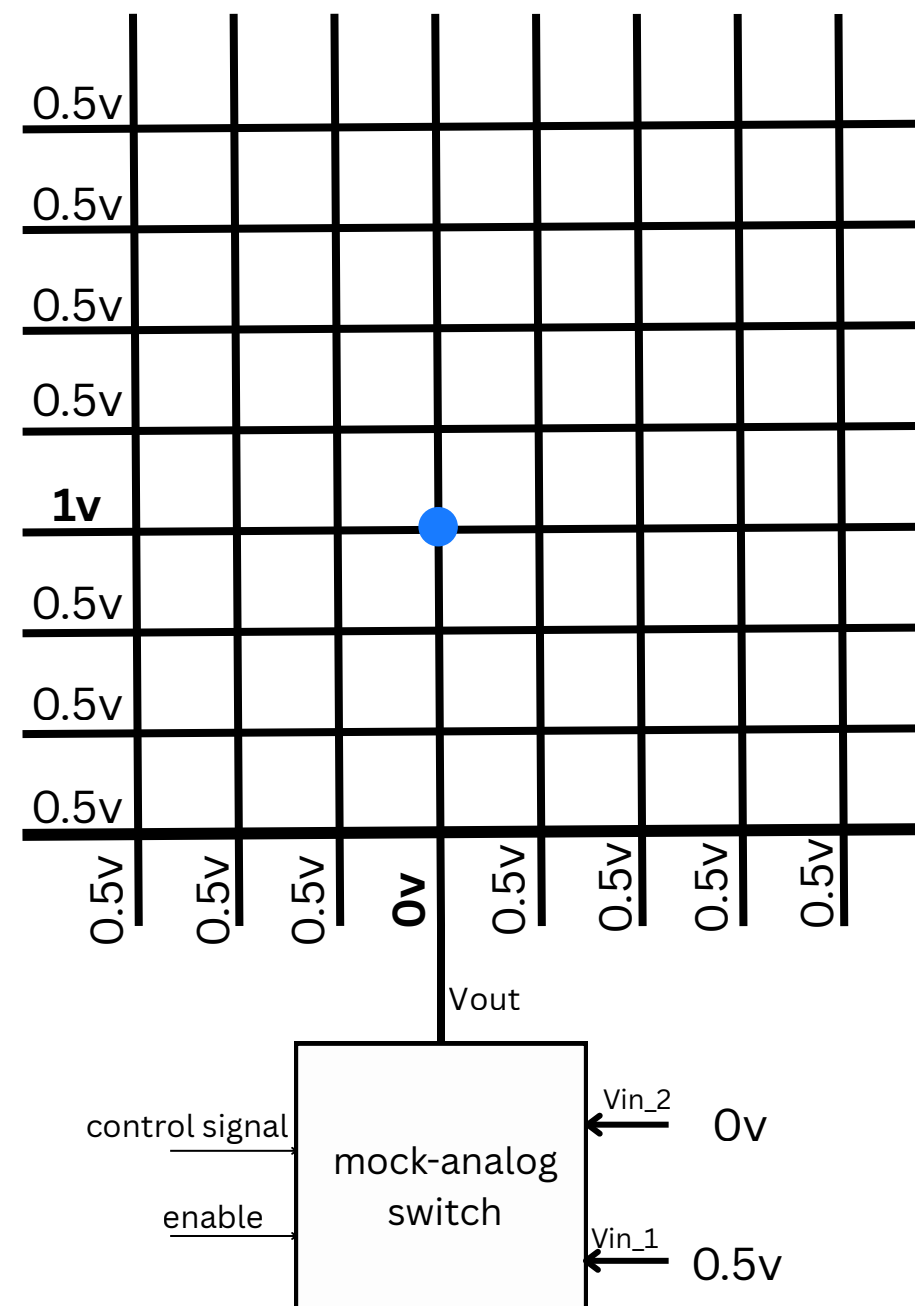
$$f_{clk}(max) = \frac{1}{max\left(\frac{t_{on}(mux_en)+t_{trans1}-t_{on}(sw_on)}{3}, \frac{t_{trans2}+t_{on}(sw_en)-t_{off}(sw_en)-pw}{n}\right)}$$

n	pw (ns)	f_clk_max (Hz)
5	50	18.52 MHz
5	100	22.73 MHz
5	250	71.43 MHz
5	400	115.38 MHz
10	50	37.04 MHz
10	100	45.45 MHz
10	250	115.38 MHz
10	400	115.38 MHz
15	50	55.56 MHz
15	100	68.18 MHz
15	250	115.38 MHz
15	400	115.38 MHz

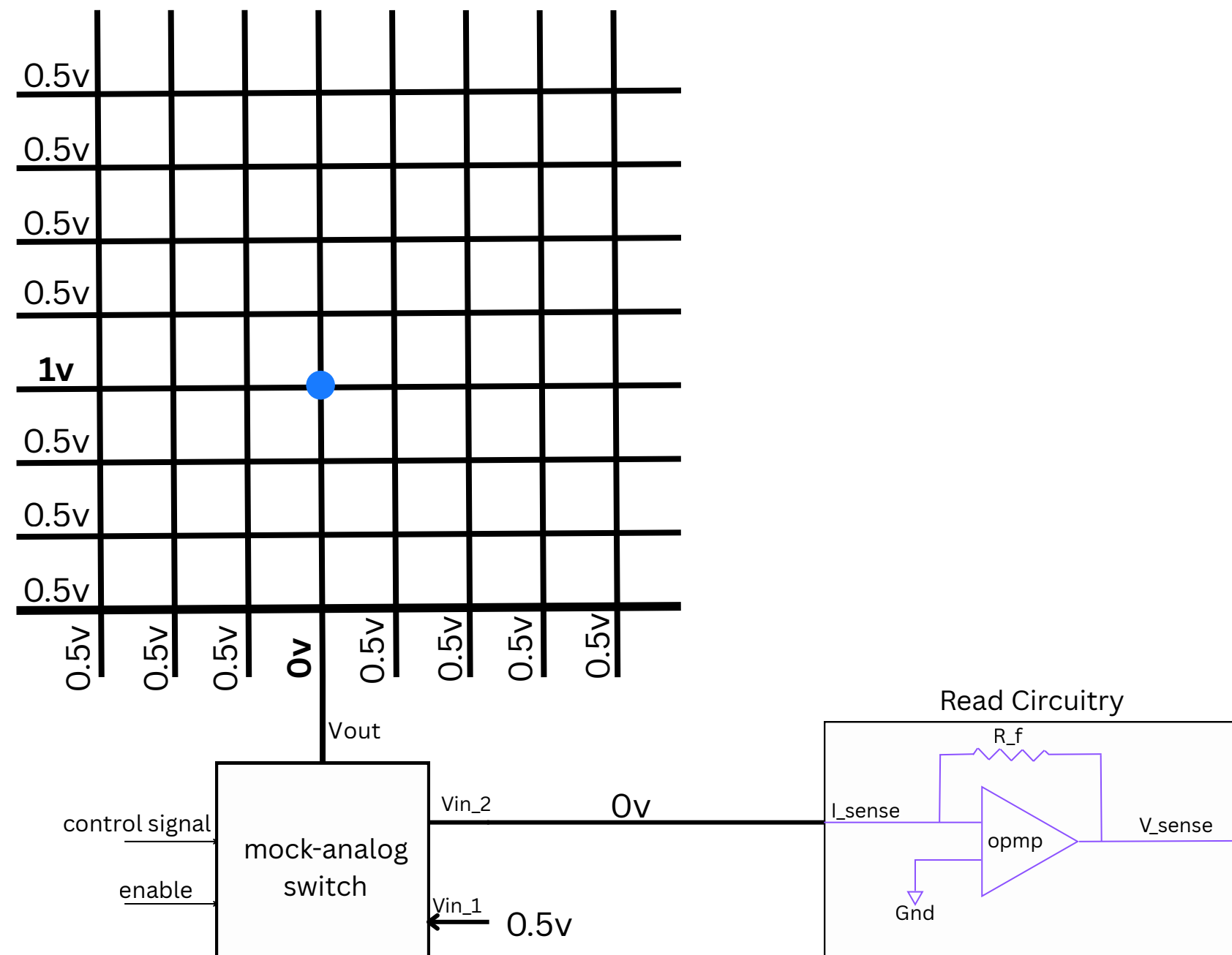
For our implementation, n = 5 and pulse width = 100ns, so the working frequency will be around 22.73 MHz.

Another solution to this problem is to use different clocks for the pulse width and the state machine. Use a slower clock for the state machine and a faster clock for the pulse width to make it 100ns wide.

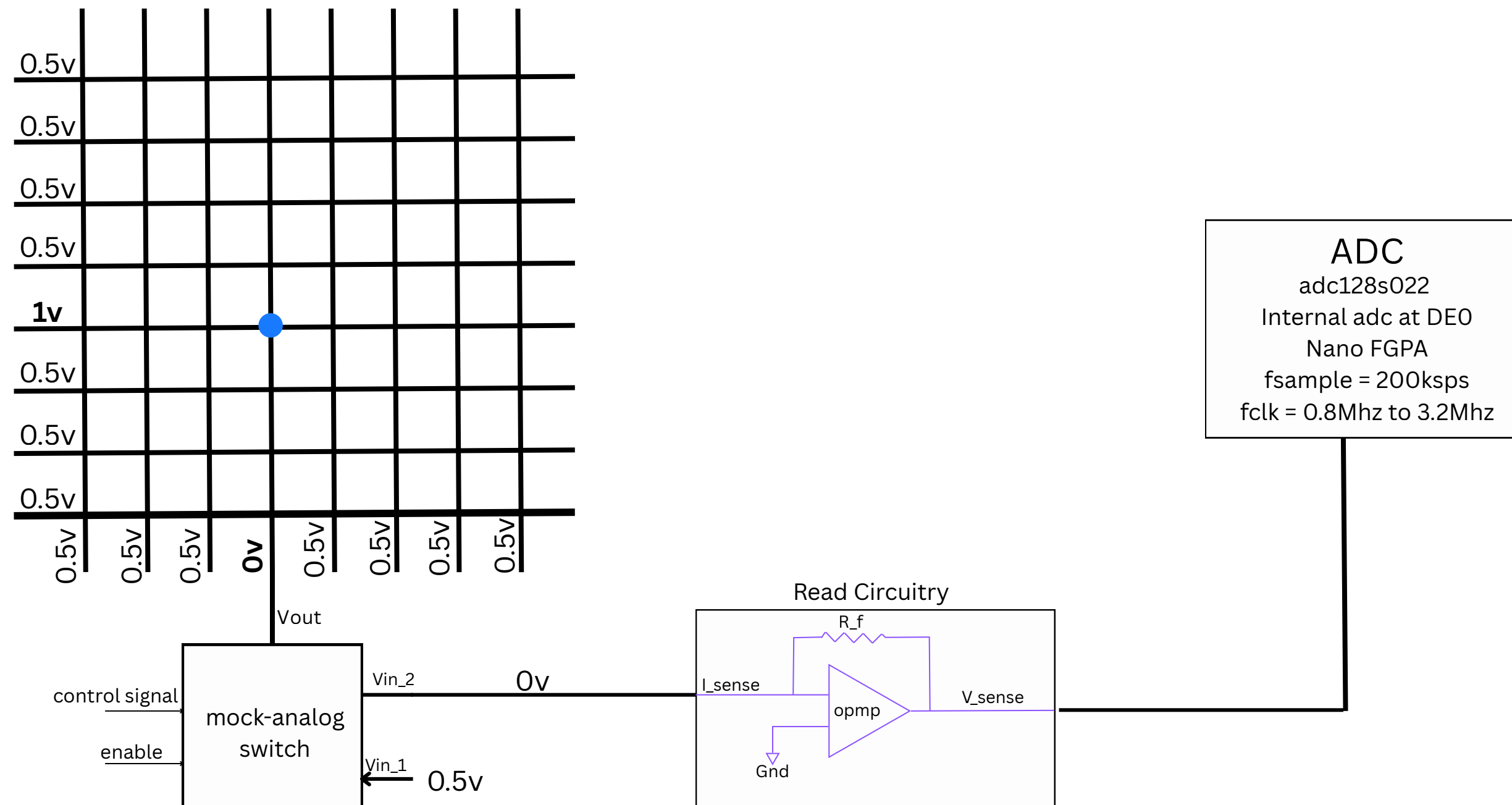
READ CIRCUITRY



READ CIRCUITRY



READ CIRCUITRY



READ CIRCUITRY

ADC128S022 is an SPI-based analog-to-digital converter that completes one conversion in 16 clock cycles.

ADC128S022 – Hold Time Requirements (for 16 SPI clock cycles):

- **At 0.8 MHz SCLK:** Clock period = $1.25\mu\text{s}$, Total hold time = $1.25\mu\text{s} \times 16 = 20\mu\text{s}$
- **At 1 MHz SCLK:** Clock period = $1\mu\text{s}$, Total hold time = $1\mu\text{s} \times 16 = 16\mu\text{s}$
- **At 2 MHz SCLK:** Clock period = $0.5\mu\text{s}$, Total hold time = $0.5\mu\text{s} \times 16 = 8\mu\text{s}$
- **At 3.2 MHz SCLK:** Clock period = $0.3125\mu\text{s}$, Total hold time = $0.3125\mu\text{s} \times 16 = 5\mu\text{s}$

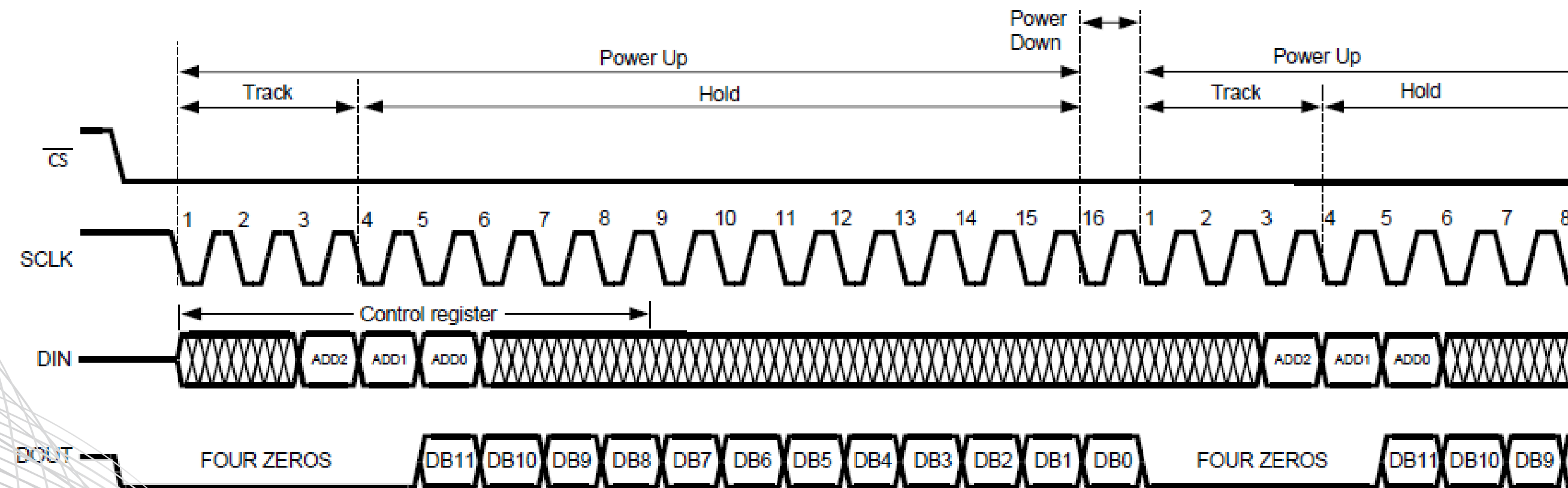
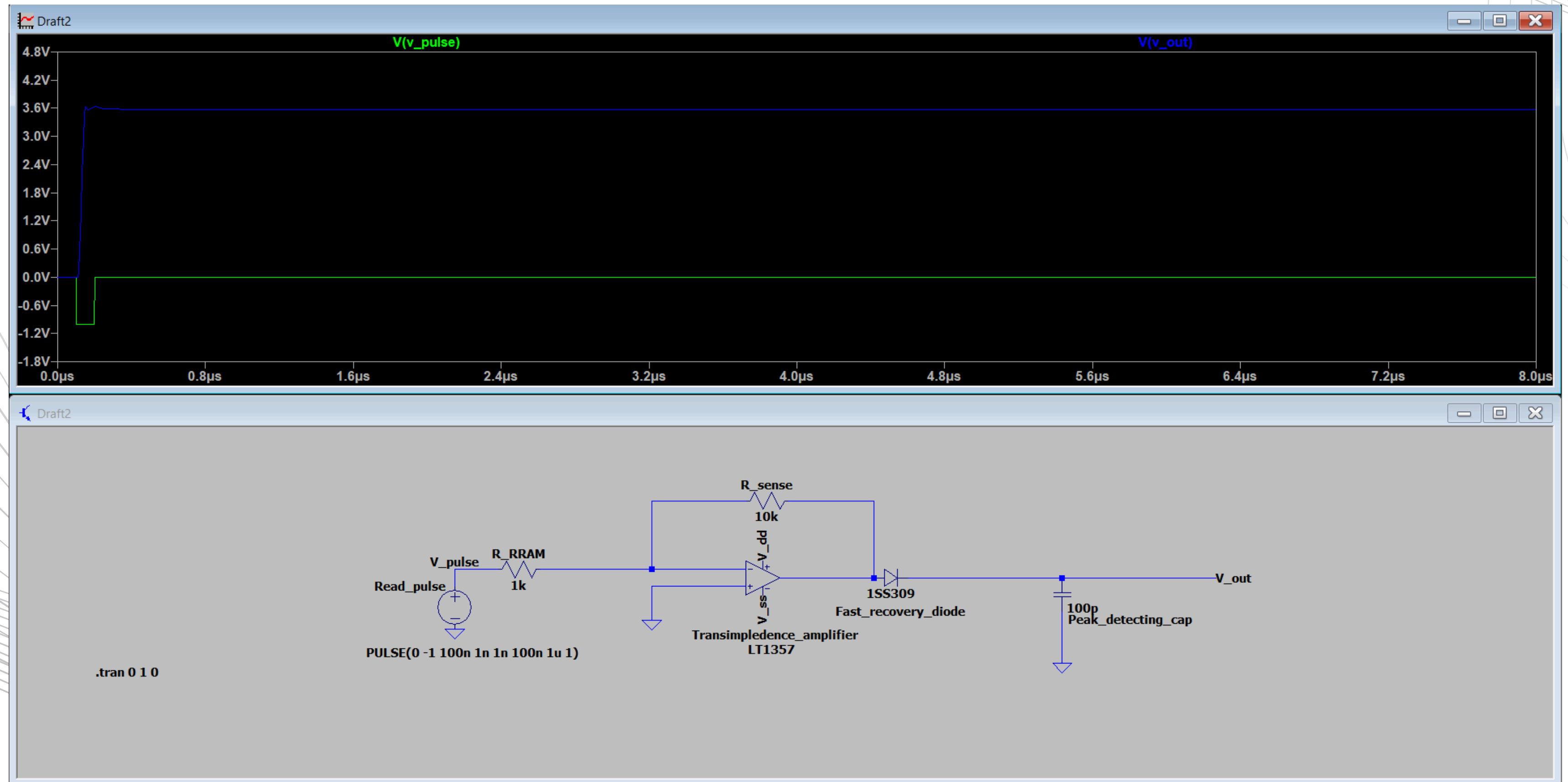
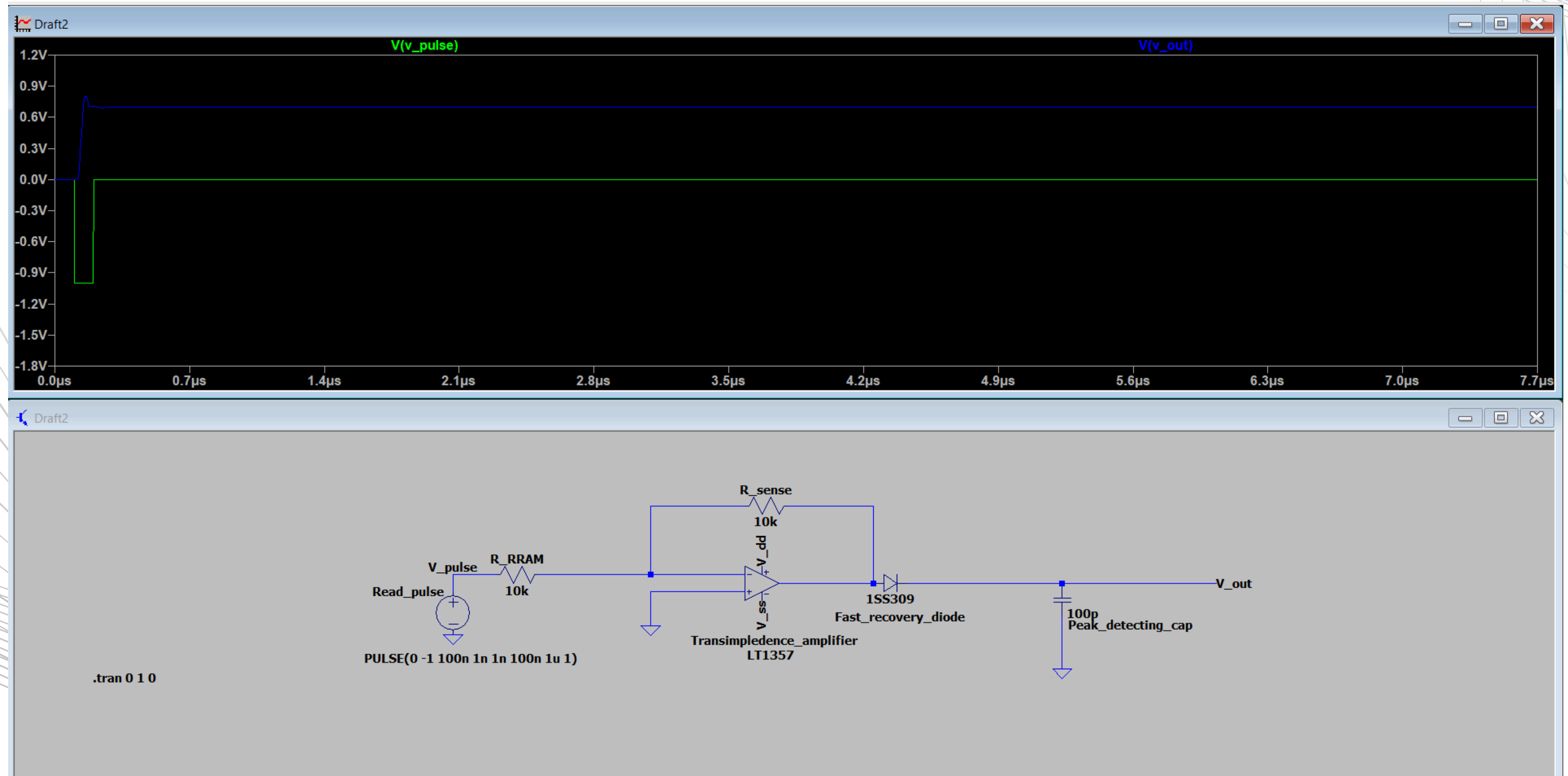


Figure 1. ADC128S022 Operational Timing Diagram

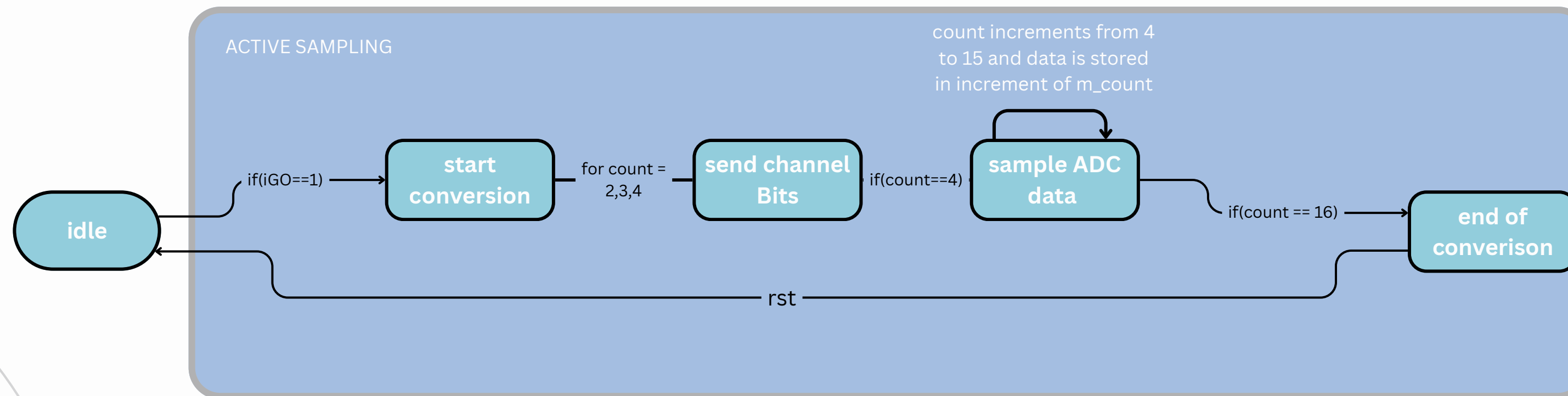
READ CIRCUITRY



READ CIRCUITRY



ADC CONTROL



- The count is an internal register that updates with every rising edge of the clock.
- The mcount is an internal register that updates with every falling edge of the clock.
- The channel bits are transmitted with each positive edge of `sclk`.
- The adc sample bit is received at each negative edge of `sclk`.

Pin Descriptions

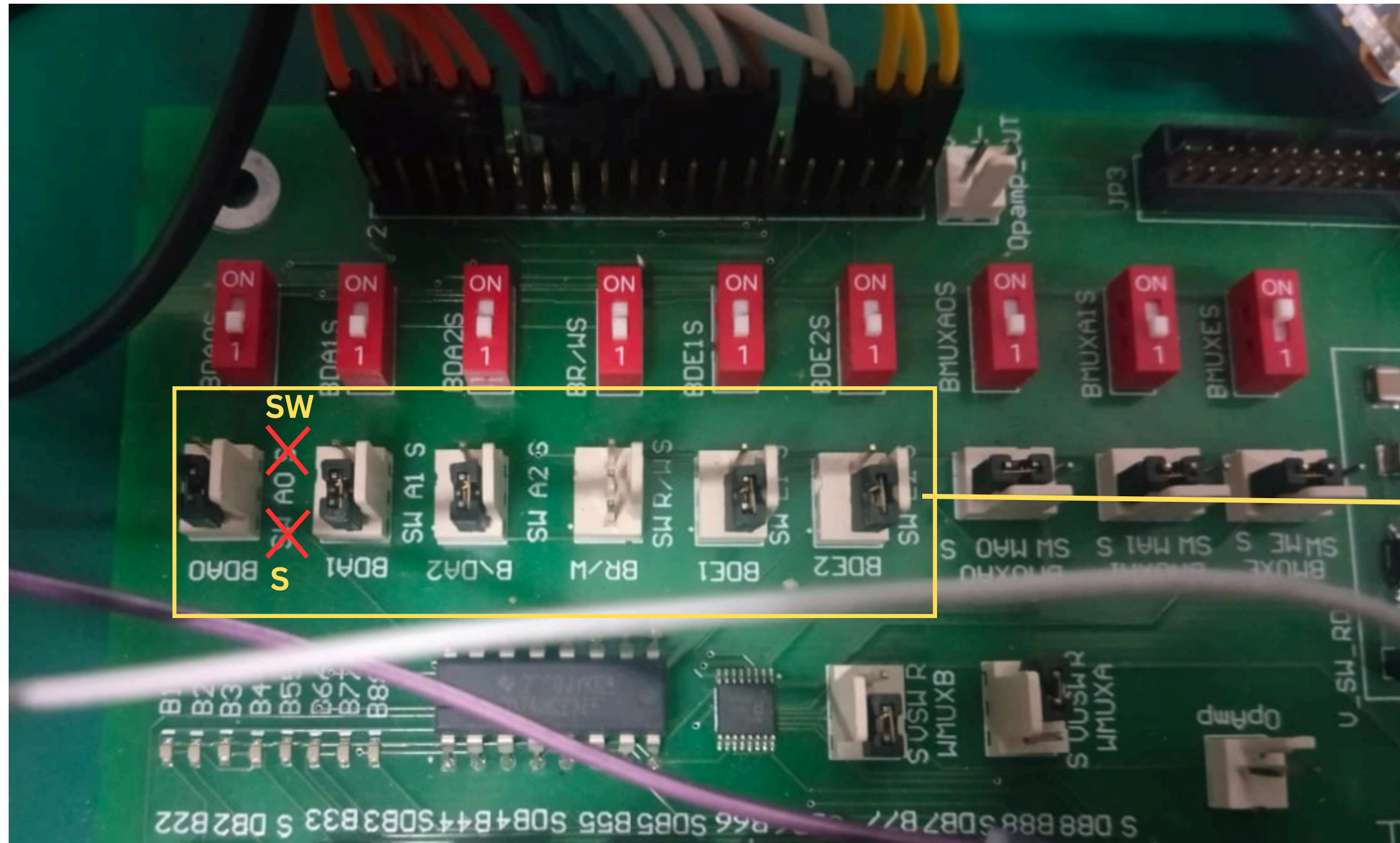
Inputs

- **iRST**: Reset signal – active-low. Resets control logic and data registers.
- **iCLK**: Main SPI clock – used for internal state timing and data shifting.
- **iCLK_n**: Inverted SPI clock – used to align channel bit output and capture cont safely.
- **iGO**: Start Conversion – a pulse to initiate a new ADC conversion cycle.
- **iCH[2:0]**: Channel Select – 3-bit input to select the ADC channel (0–7).
- **iDOUT**: Serial Data from ADC – sampled bits from the ADC device (SDO).

Outputs

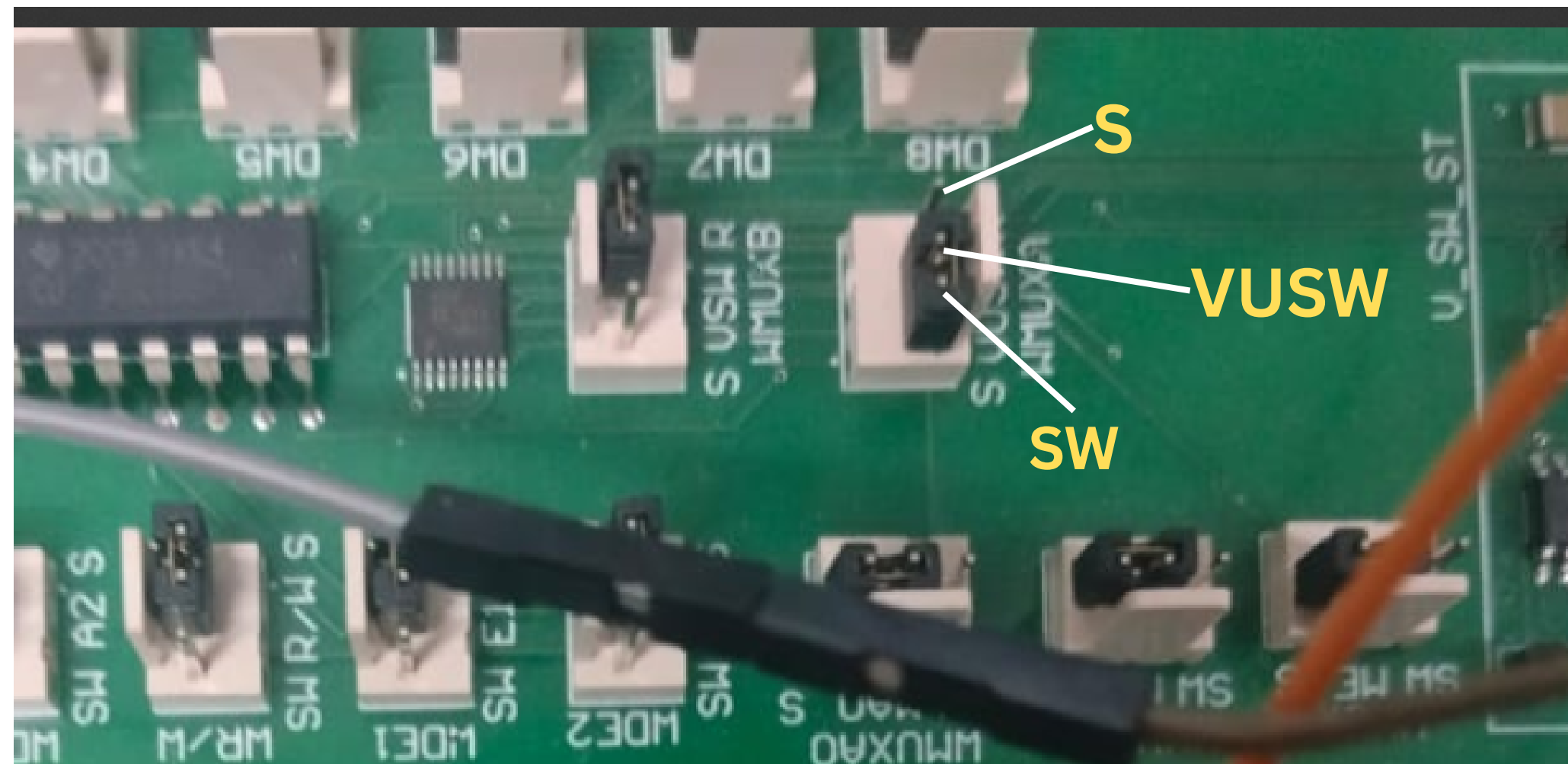
- **oADC_data[11:0]**: 12-bit Parallel Output – holds final ADC conversion result.
- **oEOC**: End of Conversion – high when conversion is complete and data is valid.
- **oDIN**: Serial Data to ADC – sends channel selection bits to ADC (SDI).
- **oCS_n**: Chip Select (Active Low) – controls ADC access (driven low during active conversion).
- **oSCLK**: SPI Clock Output – toggles with `iCLK` when conversion is active.

OBSERVATION



Same for all in these row

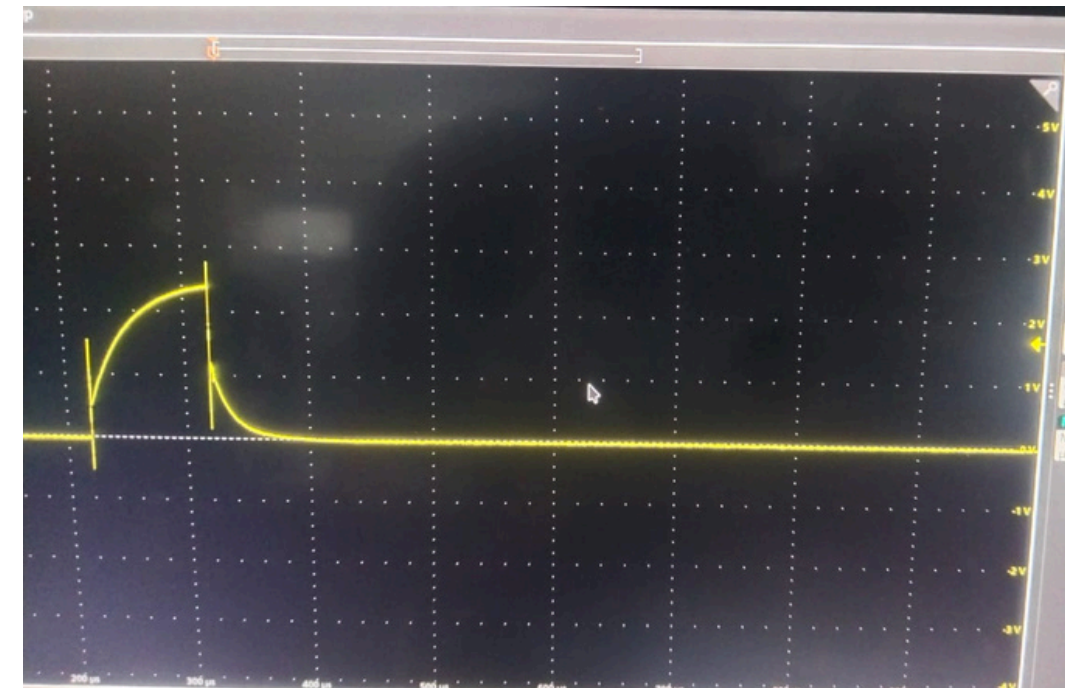
OBSERVATION



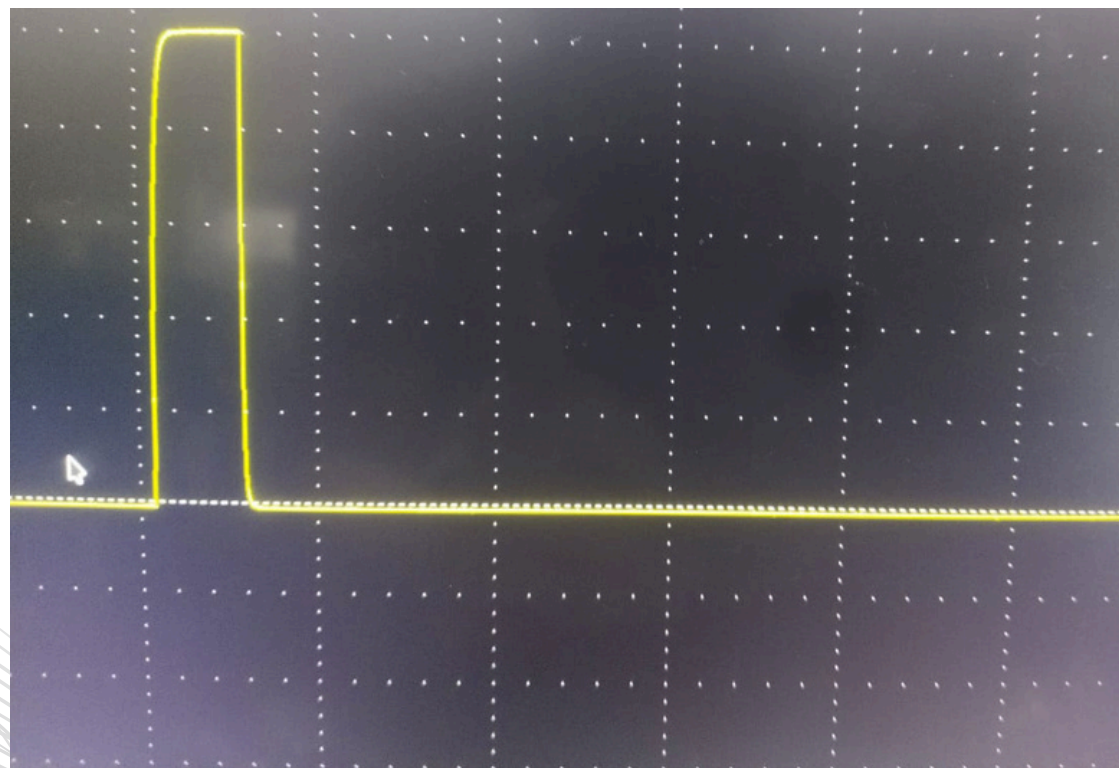
OBSERVATION



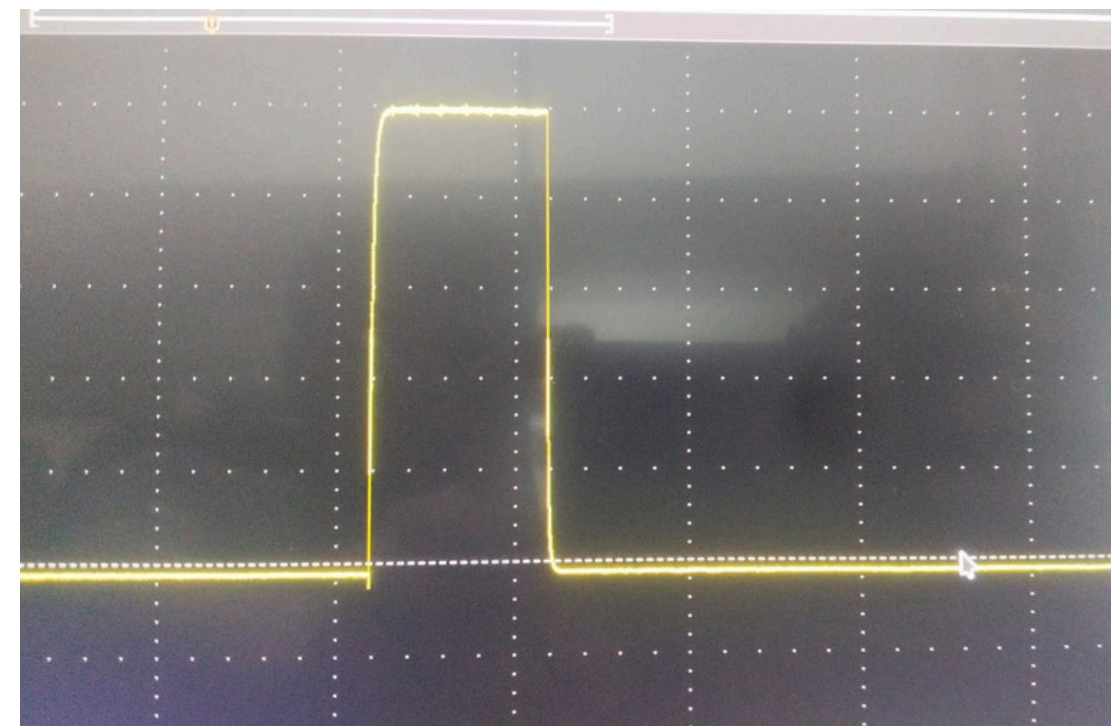
For 1MHz clk with 5us pulse width



For 100k Hz clk with 50us pulse width



For 10k Hz clk with 500us pulse width



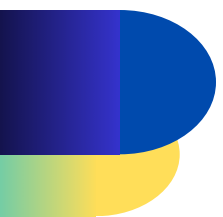
For 5k Hz clk with 1ms pulse width

OBSERVATION

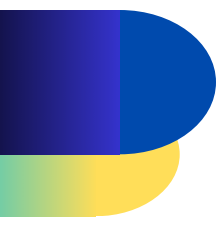
Observation Table

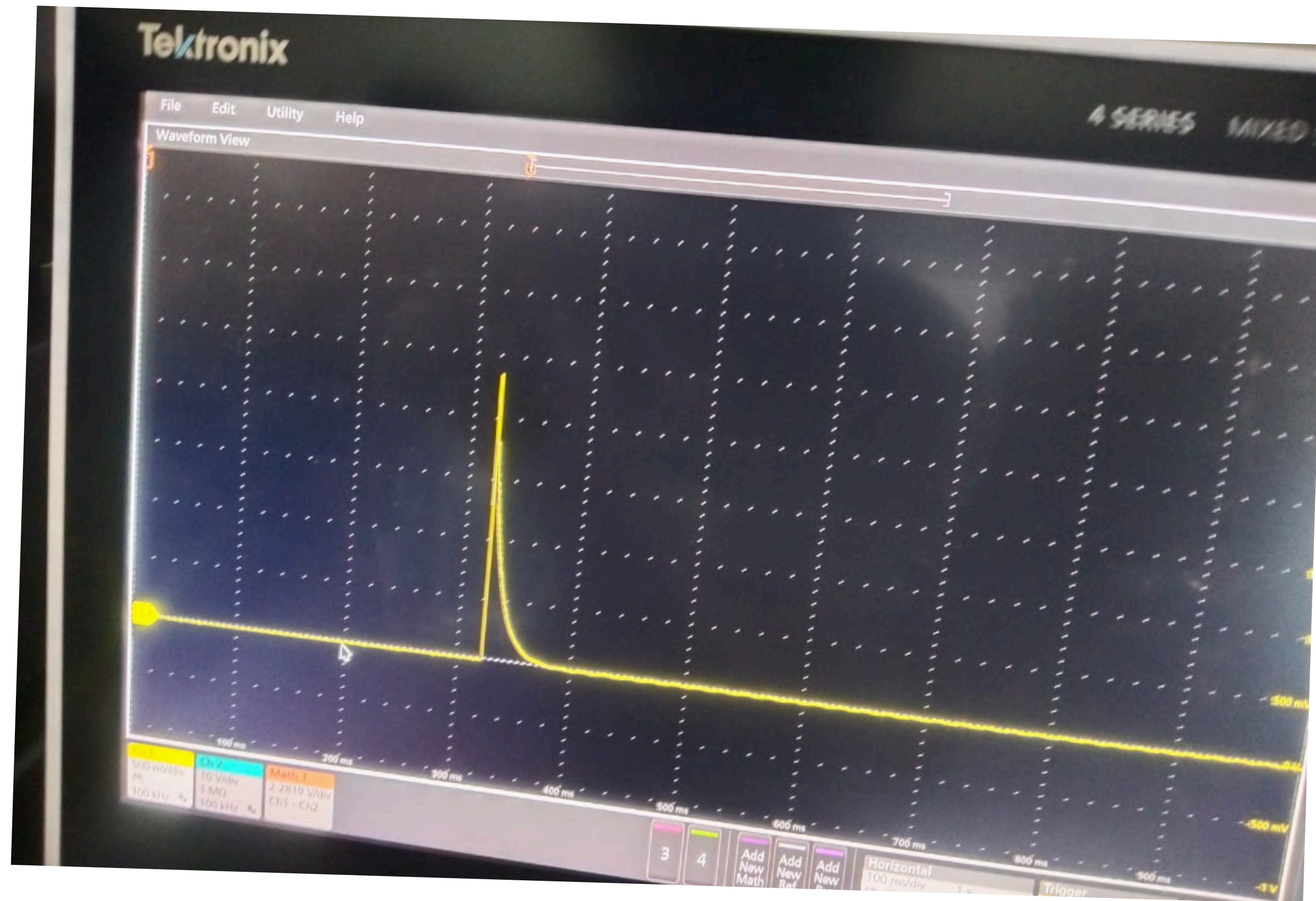
Frequency	Pulse width	V observed	Observation
1MHz	5us	Peak of 2.2V and avg over 0us to 5us is 1.75v	Severe ringing, overshoot, underdamped transitions
100Khz	50us	Peak of 2.2v and avg over whole is 1.44v but avg over 24us to 50us (that is rise time to settling time) is 1.78v	Noticeable overshoot and slow return to baseline
10 Hhz	500us	Consistent peak and average of 2.2 v	Cleaner waveform
5 KHz	1ms	Consistent peak and average of 2.22 v	Smooth transition

WHAT NEXT?

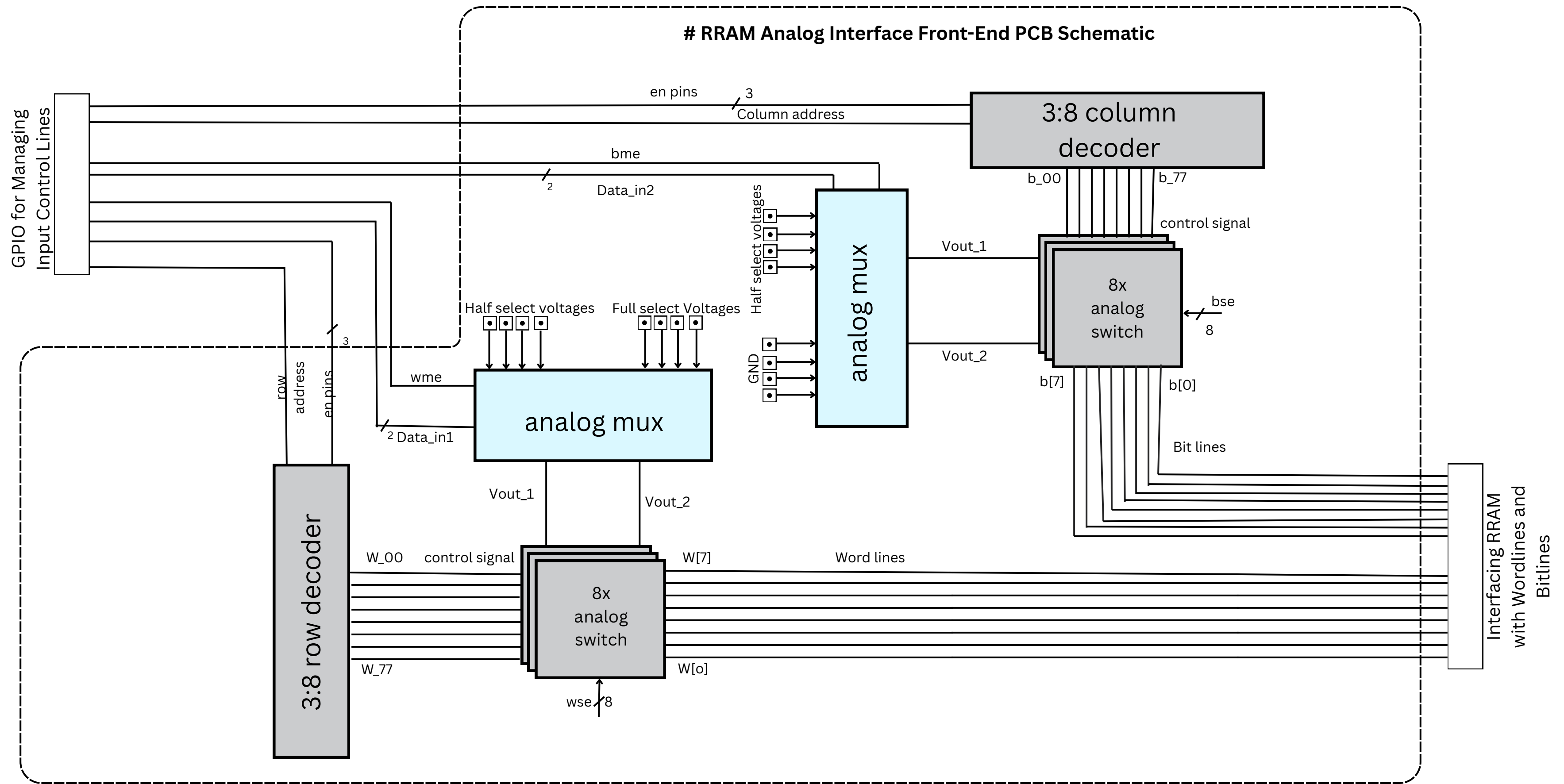


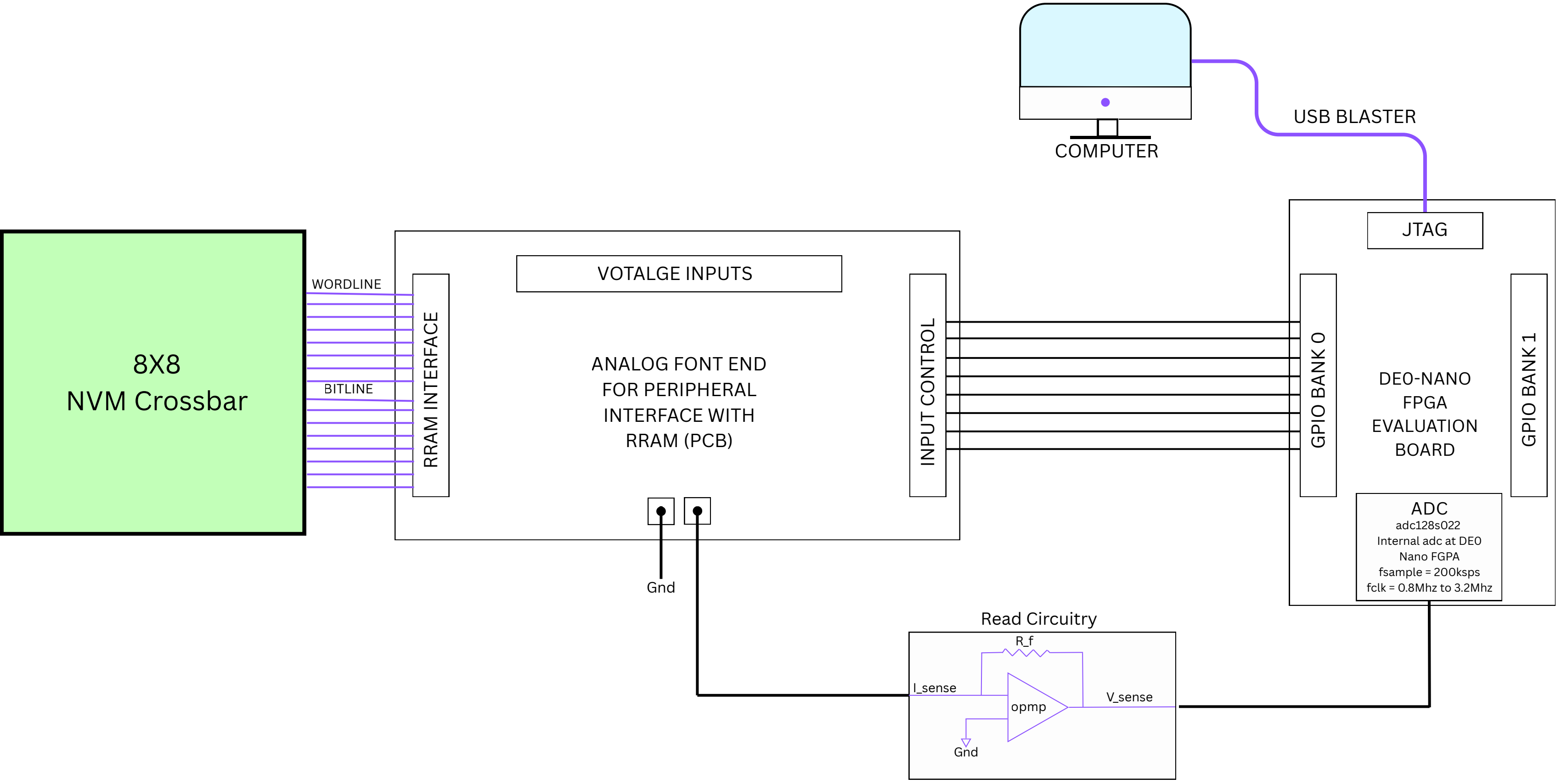
THANK YOU

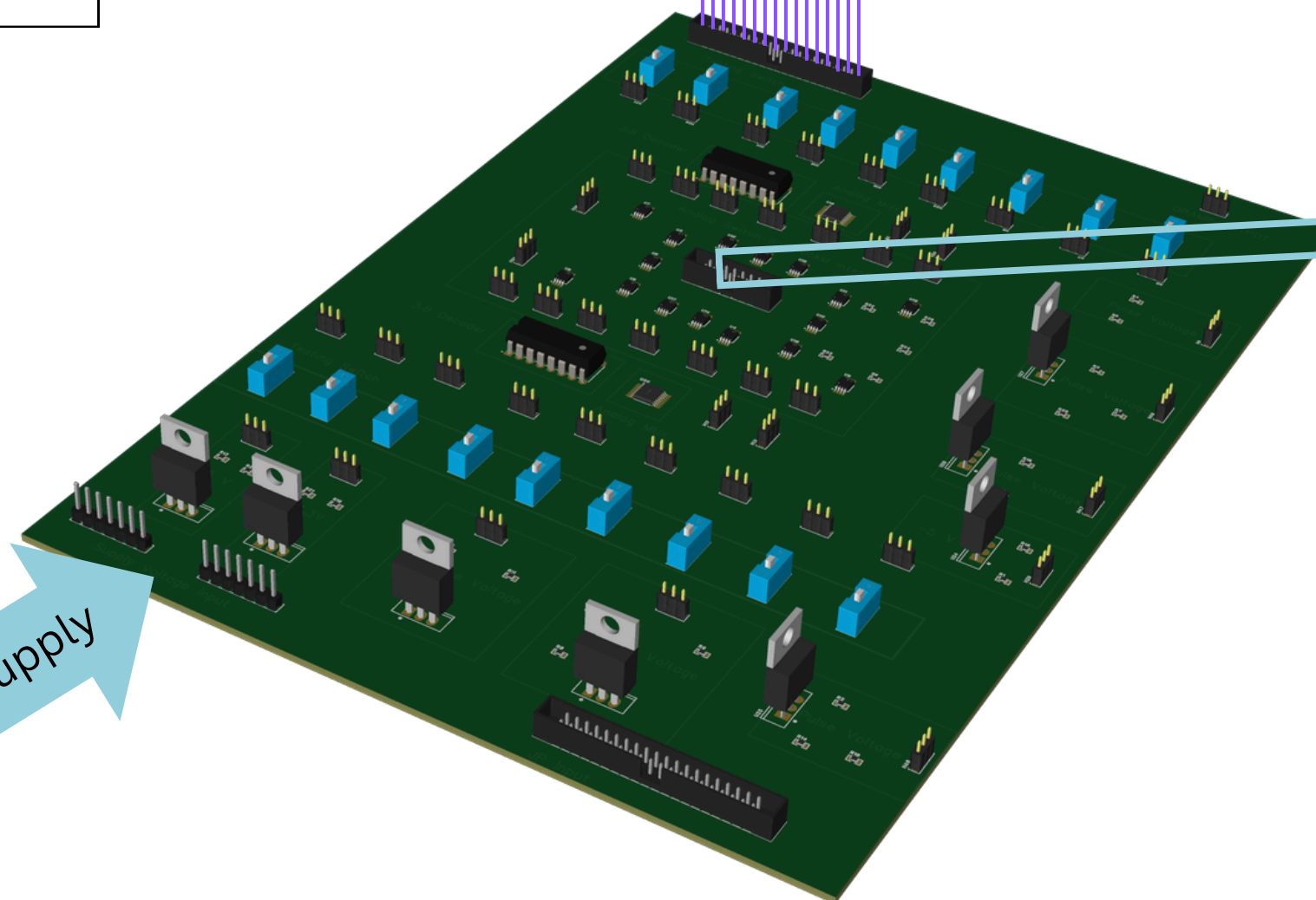
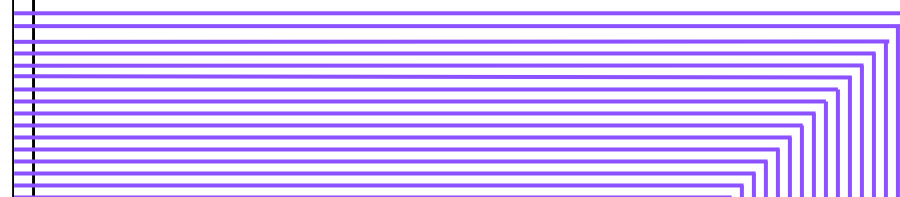
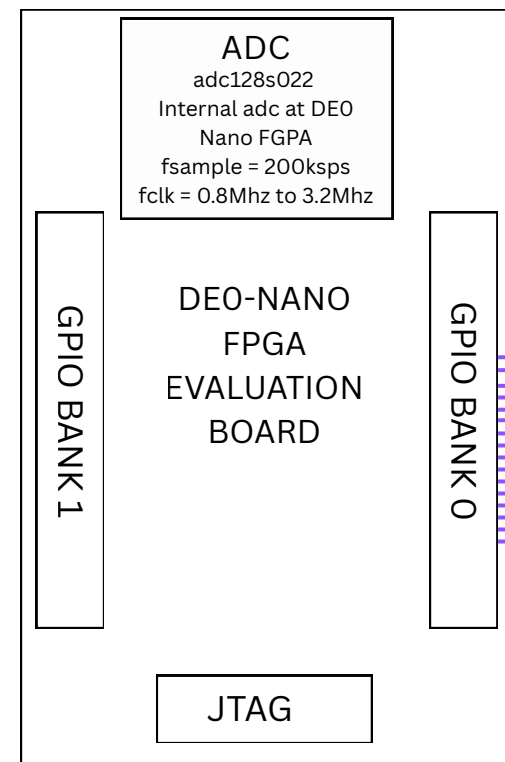




For 50MHz clk with 5ns pulse width







Tektronix 4 Series Mixed Signal Oscilloscope



Keithley 3-channel Power supply